HCL-ASSESSMENT - 2

NAME : CHALAMSETTY NIVAS KRISHNA
DEPARTMENT : CSE-AIML
USN : 23BTRCL173

HCLID : HCLTFP2262742

Datebase called EnterpriseDB

Table Employess

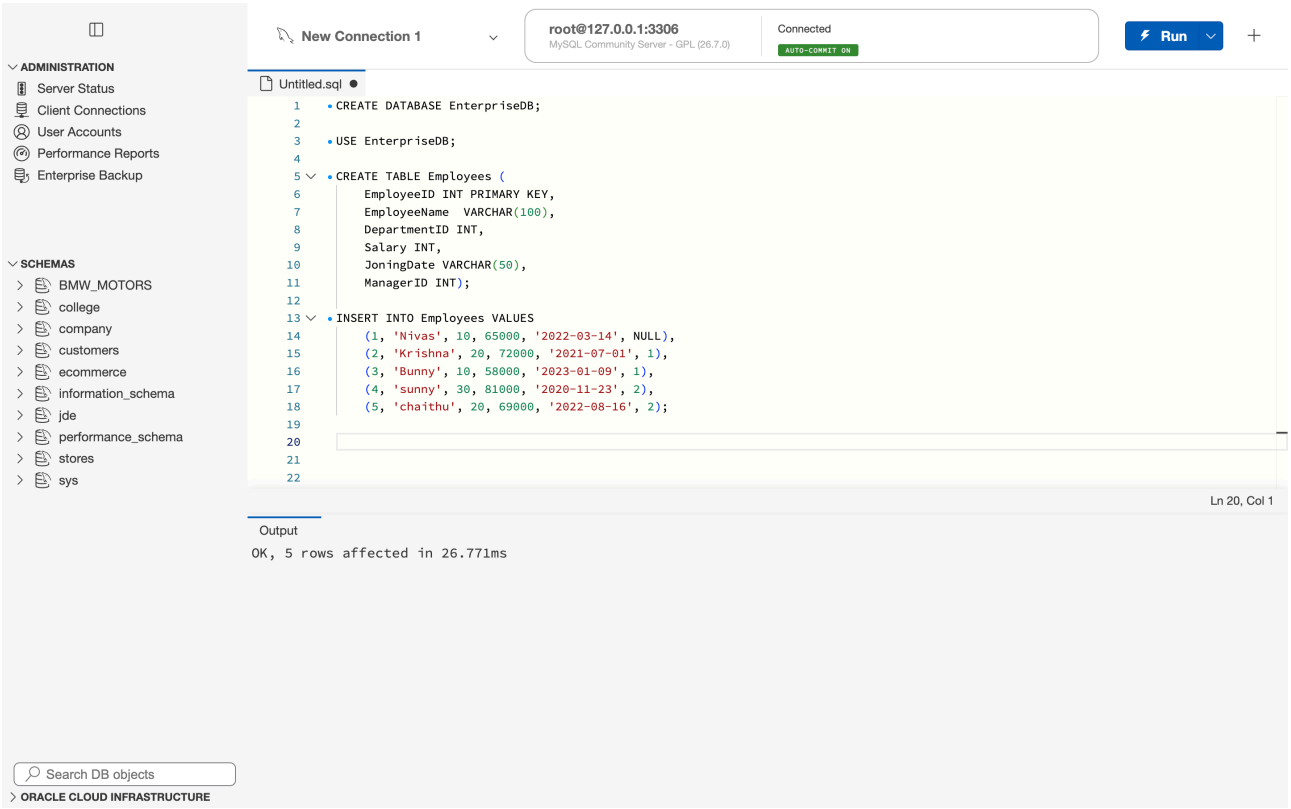
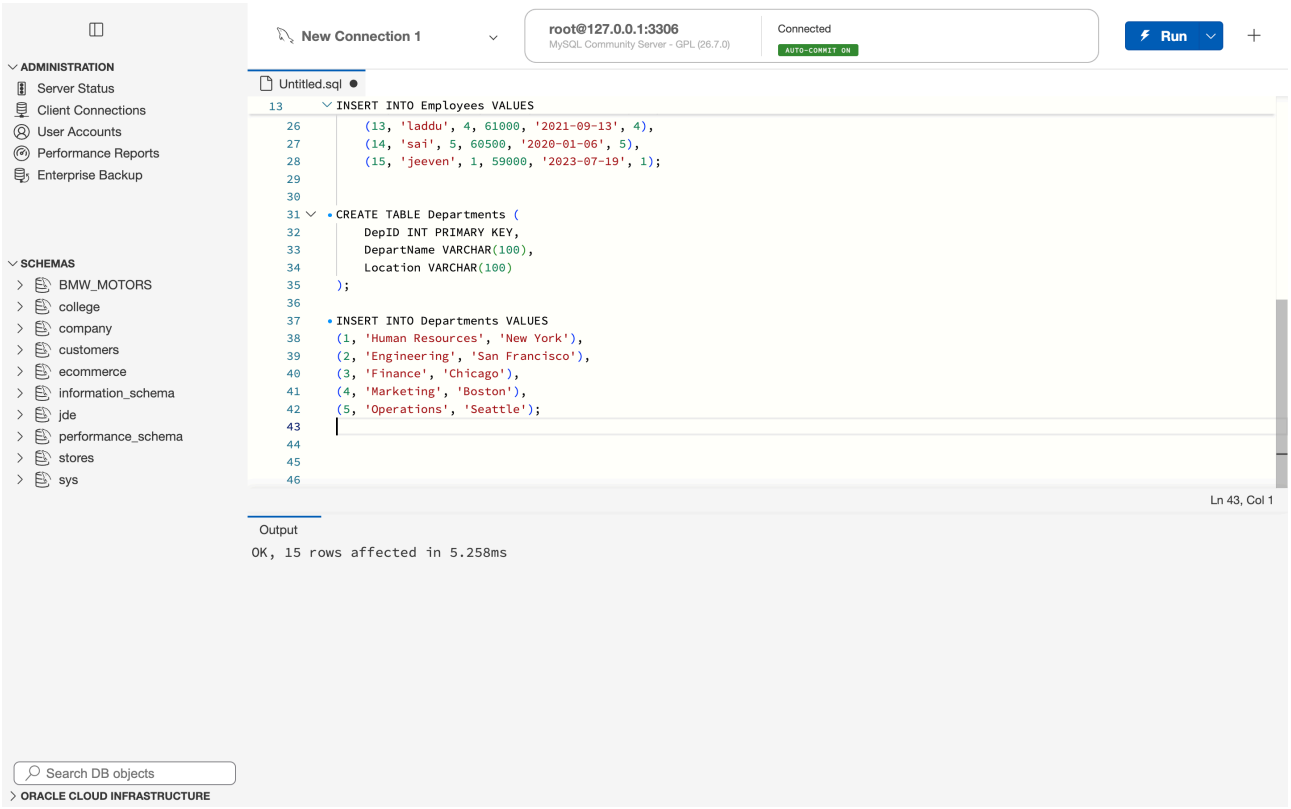
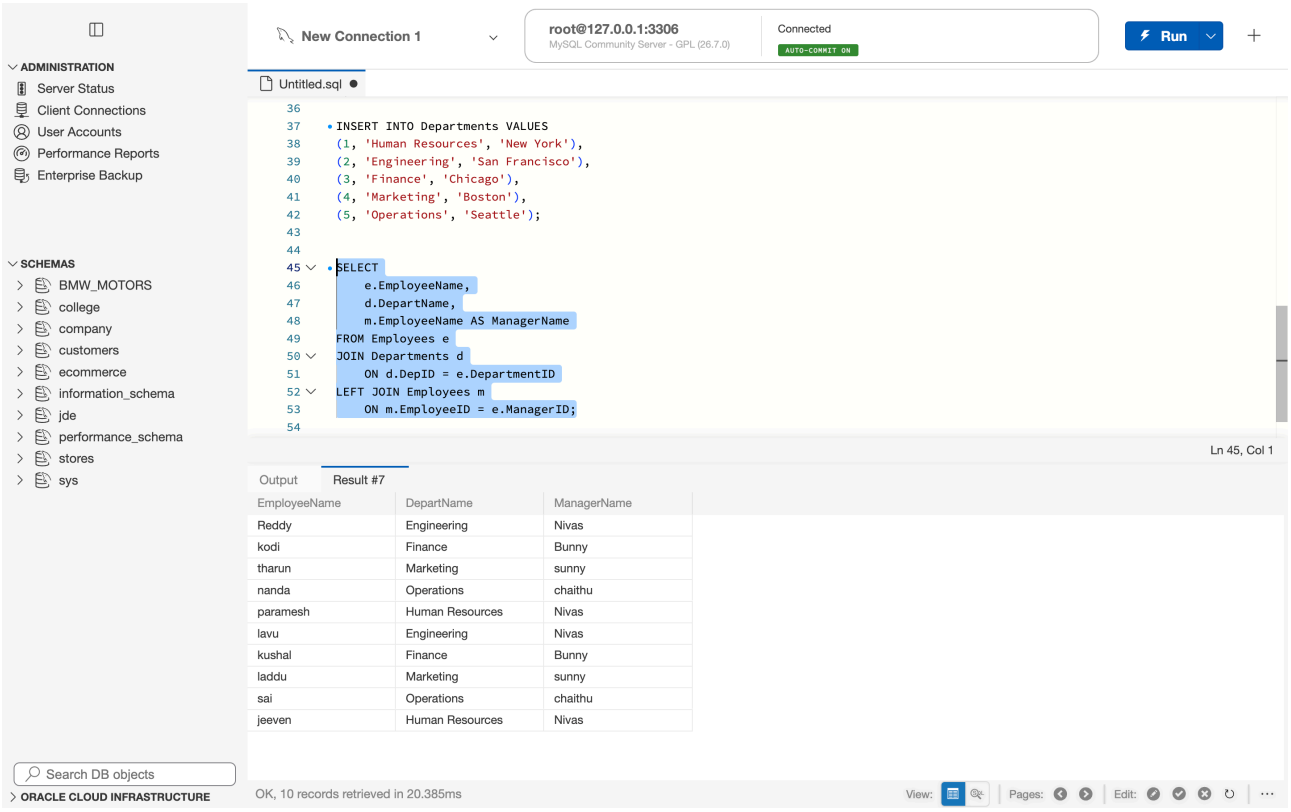


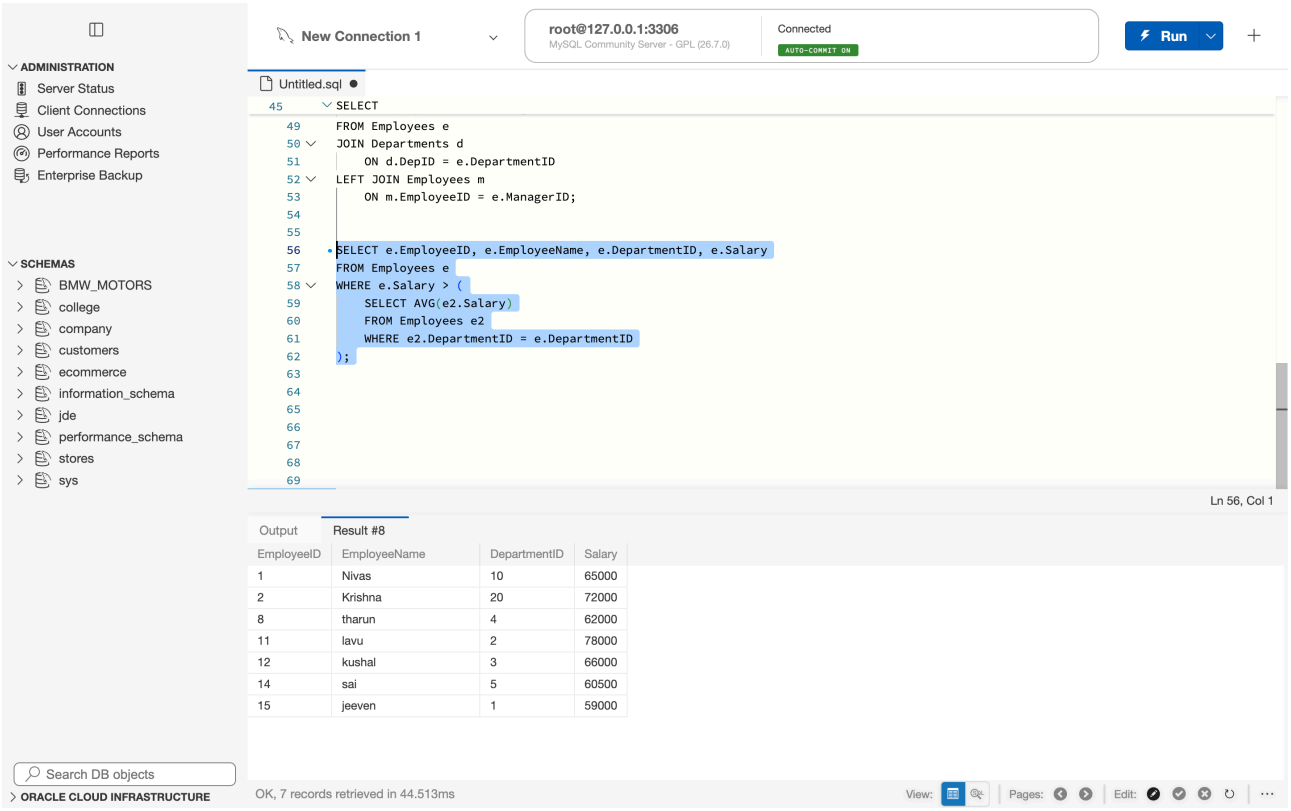
Table Departments



Employee names, department names, and manager names using joins



Employees earning more than their departments average salary



Second-highest distinct salary

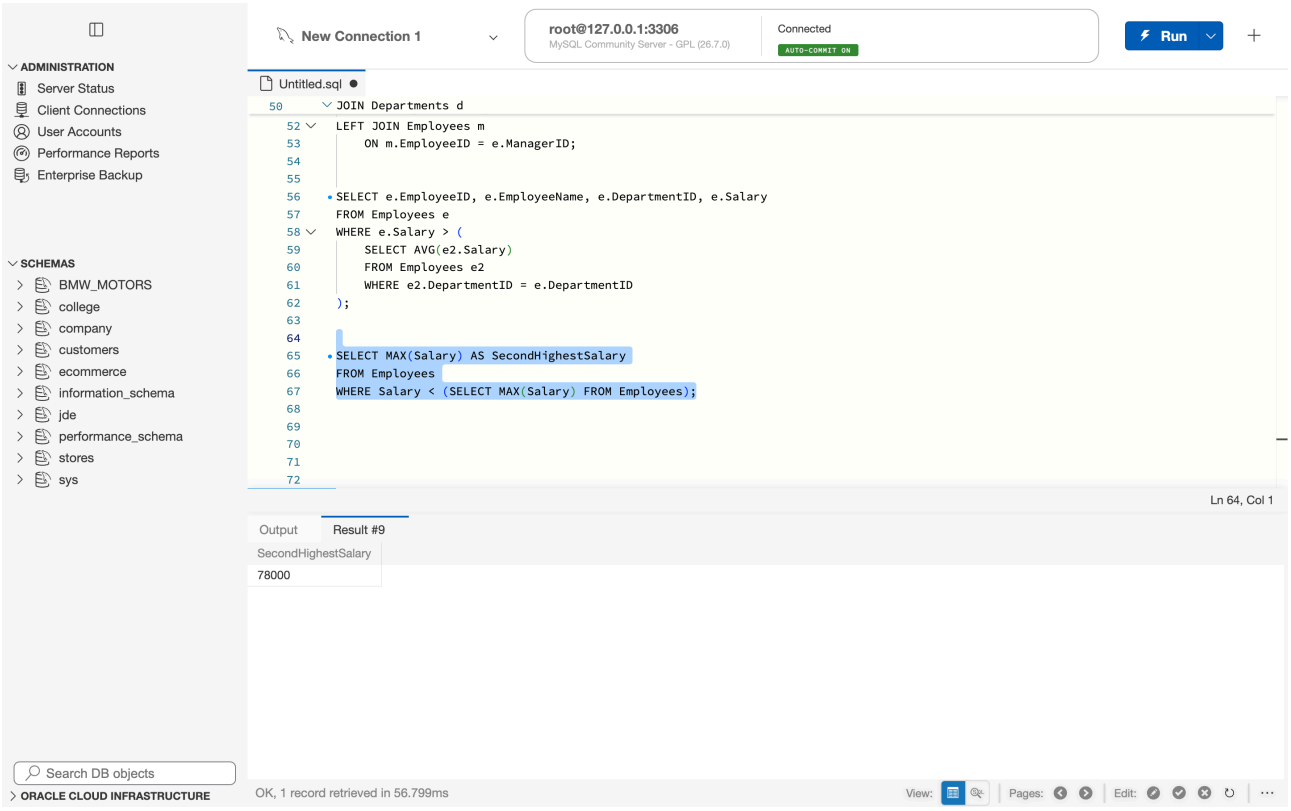
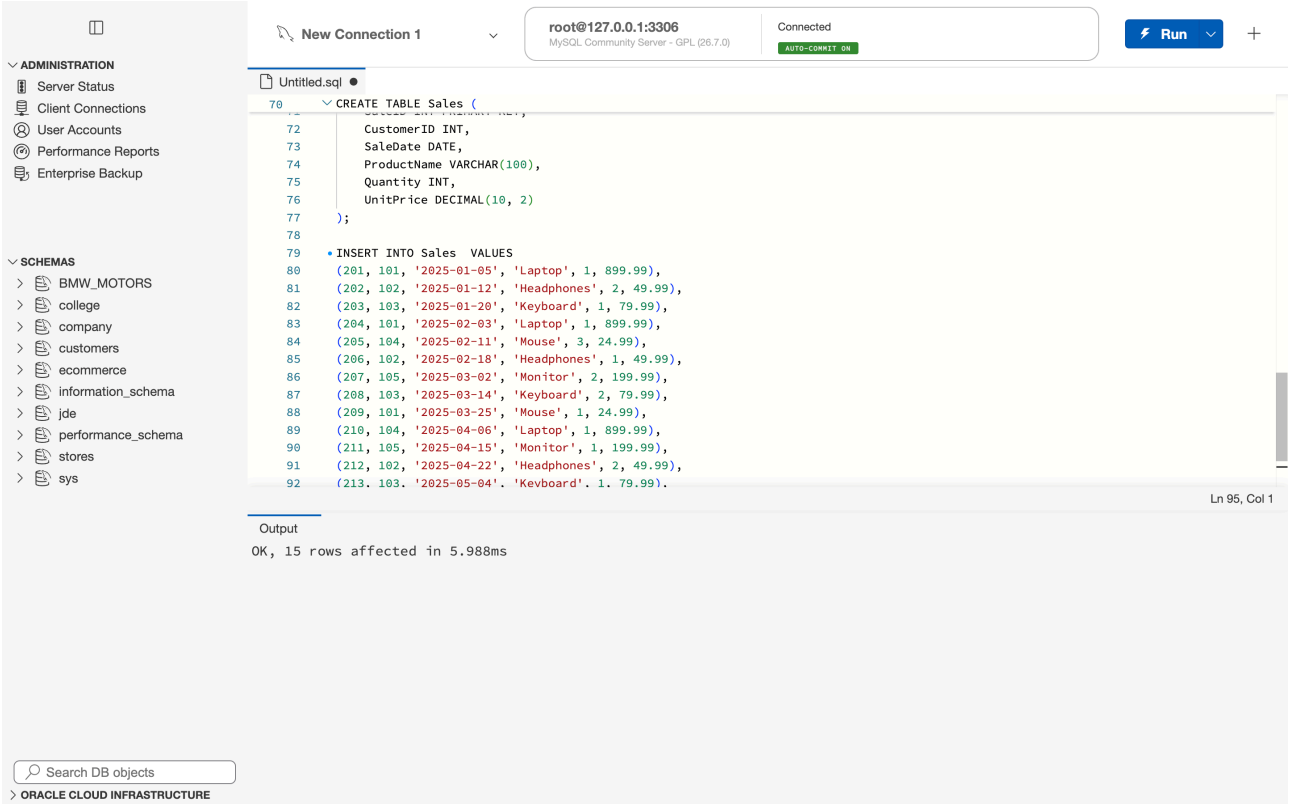
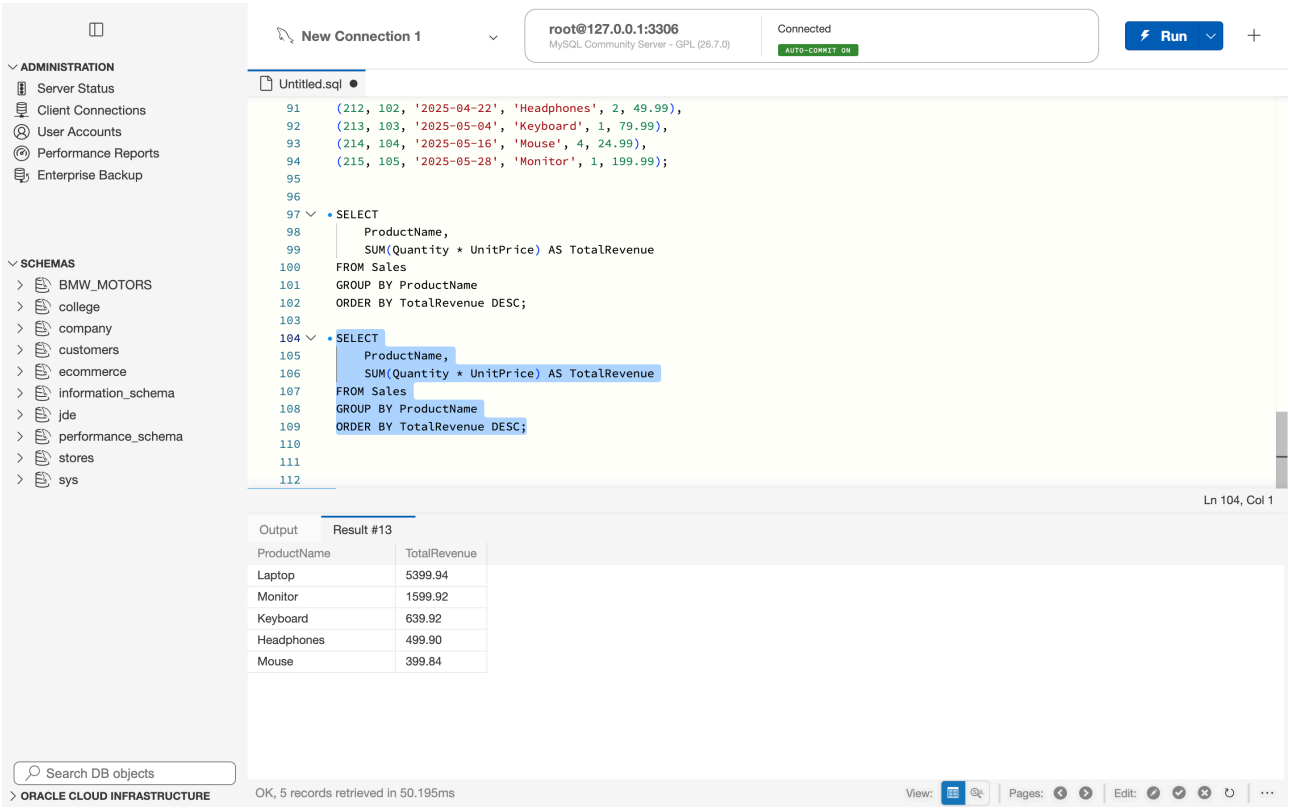


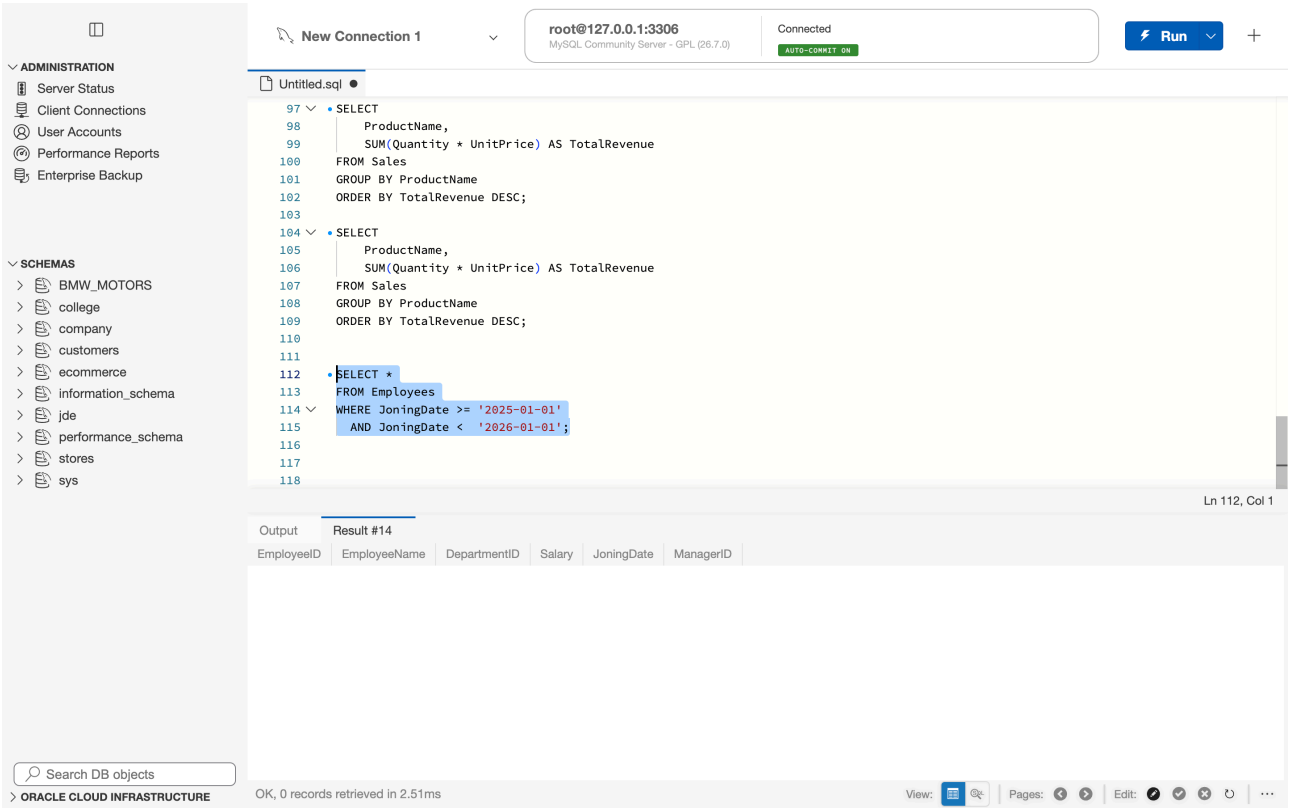
Table Sales and inserted values 15 records



Total revenue by product



Rewrite for index usage



SQL TroubleShooting

1. Explaing how an execution plan helps identify expensive operations in a query ?

ANS : an execution plan shows how the database will retrieve and process rows for a query. It helps locate costly steps such as full table scans, large sorts, or joins that process many rows and shows that indexes are being used. Use it to focus optimization on the operations in the most work.

2. Recommend an appropriate index strategy for queries filtering employees by DepartmentID and Salary in details ?

ANS :

```
117
118    CREATE INDEX idx_employees_department_salary
119    ON Employees (DepartmentID, Salary);
120
```

This can help the database narrow rows by department and then salary. The column order matters: put the column used most often as an equality filter first. Check the execution plan to confirm the index helps; indexes also add storage and slow inserts or updates slightly.

3. Write a query to identify duplicate customer email addresses and display only addresses occurring more than once

ANS : to identify duplicate customer email address and to display only occurring more than one

```
121    SELECT Email, COUNT(*) AS Occurrences
122    FROM Customers
123    WHERE Email IS NOT NULL
124    GROUP BY Email
125    HAVING COUNT(*) > 1;
126
```

Log Analysis

Assume /var/log/application.log contains millions of log entries in the following
format:
2026-10-28 10:15:23 ERROR payment-service Transaction failed

1. Find all lines containing ERROR

ANS :

```
[ec2-user@ip-172-31-31-162 ~]$ sudo grep "ERROR" /var/log/application.log
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 10:22:01 ERROR auth-service Login attempt failed
2026-10-28 11:05:00 ERROR payment-service Connection timeout
2026-10-28 11:12:34 ERROR payment-service Transaction failed
[ec2-user@ip-172-31-31-162 ~]$
```

2. Display the 20 most recent error entries in the file

ANS:

```
[ec2-user@ip-172-31-31-162 ~]$
[ec2-user@ip-172-31-31-162 ~]$ sudo grep "ERROR" /var/log/application.log | tail -n 20
2026-10-28 10:15:23 ERROR payment-service Transaction failed
2026-10-28 10:22:01 ERROR auth-service Login attempt failed
2026-10-28 11:05:00 ERROR payment-service Connection timeout
2026-10-28 11:12:34 ERROR payment-service Transaction failed
[ec2-user@ip-172-31-31-162 ~]$
```

i-08dc74f76d3dfaf97 (web computer 1)

3. Count the total number of error entries.

ANS :

```
2026-10-28 11:12:34 ERROR payment-service Transaction failed
[ec2-user@ip-172-31-31-162 ~]$ sudo grep -c "ERROR" /var/log/application.log
4
```

4. Extract error entries for a specified date into a separate file

ANS :

```
[ec2-user@ip-172-31-31-162 ~]$ sudo grep "2026-10-28.*ERROR" /var/log/application.log > todays_errors.txt
```

5. Explain how you would calculate the number of errors **occurring** in each hour.

ANS :

```
[ec2-user@ip-172-31-31-162 ~]$ sudo grep "ERROR" /var/log/application.log | cut -c 1-13 | uniq -c
      2 2026-10-28 10
      2 2026-10-28 11
[ec2-user@ip-172-31-31-162 ~]$
```

Permissions and System Troubleshooting

1. Create a group named finance and a report file, assuming you have the required privileges

ANS : To create the group and file, use sudo groupadd finance and touch report.txt

```
[ec2-user@ip-172-31-31-162 ~]$ sudo groupadd finance && touch report.txt
```

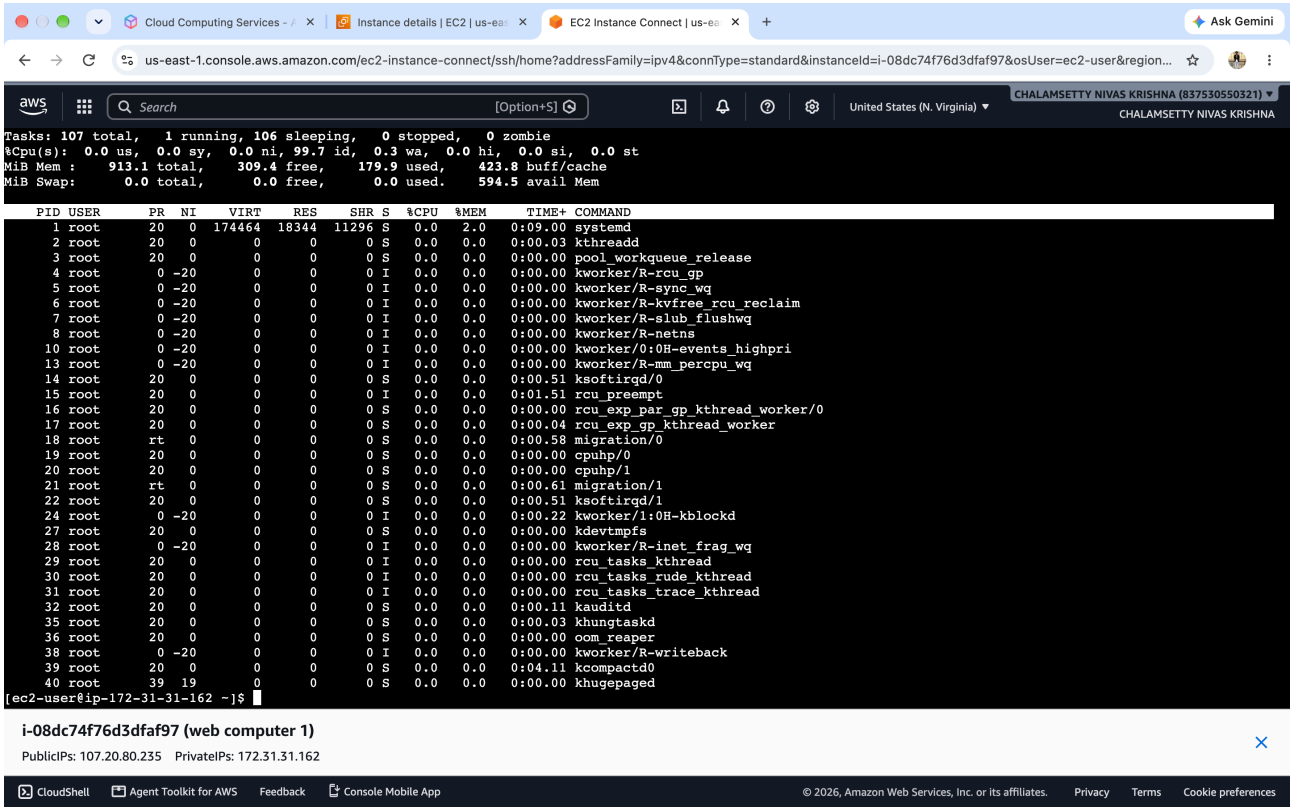
2. Set permissions so the owner can read and write, the finance group can read, and other users have no access

ANS : To set the permissions so the owner can read and write, the group can read, and others have no access, use chmod 640 report.txt and sudo chgrp finance report.txt.

```
ec2-user@ip-172-31-31-162 ~]$ chmod 640 report.txt && sudo chgrp finance report.txt
```

3. Identify the processes consuming the most CPU

ANS : To identify processes consuming the most CPU, you can use the top command.



4. Display memory usage and filesystem disk utilization

ANS : To display memory usage and filesystem disk utilization, use free -h and df -h.

```
[ec2-user@ip-172-31-31-162 ~]$ free -h
               total        used        free      shared  buff/cache   available
Mem:           913Mi       178Mi       310Mi       0.0Ki       423Mi       595Mi
Swap:             0B          0B          0B
[ec2-user@ip-172-31-31-162 ~]$
```

5. Find files larger than 100 MB under a specified application directory

ANS : Finally, to find files larger than 100 MB in a specific directory, use find /path/to/app -type f -size +100M

```
[ec2-user@ip-172-31-31-162 ~]$ find /path/to/directory -type f -size +100M
```

Bash Scripting

First we need to create the Bash script named backup_check.sh

nano backup_check.sh

1. Accepts source and destination directories as command-line arguments

ANS:

```
if [ "$#" -ne 2 ]; then
    echo "Error: Usage: $0 <source_directory> <destination_directory>"
    exit 1
fi

SOURCE_DIR="$1"
DEST_DIR="$2"
```

2. Validates that both arguments are provided.

ANS:

```
if [ "$#" -ne 2 ]; then
    echo "Error: Usage: $0 <source_directory> <destination_directory>"
    exit 1
fi

SOURCE_DIR="$1"
DEST_DIR="$2"
```

3. Checks whether the source directory exists.

```
# 3. Check whether the source directory exists
if [ ! -d "$SOURCE_DIR" ]; then
    echo "Error: Source directory '$SOURCE_DIR' does not exist."
    exit 1
fi
```

4. Creates the destination directory if it does not exist.

```
# 4. Create the destination directory if it does not exist
if [ ! -d "$DEST_DIR" ]; then
    echo "Notice: Destination directory '$DEST_DIR' does not exist. Creating it now..."
    mkdir -p "$DEST_DIR"
    if [ $? -ne 0 ]; then
        echo "Error: Failed to create destination directory."
        exit 1
    fi
fi
```

5. Creates a timestamped compressed archive of the source directory.

```
# 5. Create a timestamped compressed archive
TIMESTAMP=$(date +"%Y%m%d_%H%M%S")
ARCHIVE_NAME="backup_$TIMESTAMP.tar.gz"
ARCHIVE_PATH="$DEST_DIR/$ARCHIVE_NAME"

echo "Creating backup of '$SOURCE_DIR'..."
tar -czf "$ARCHIVE_PATH" -C "$(dirname "$SOURCE_DIR")" "$(basename "$SOURCE_DIR")"
```

6. Displays a success message if the backup succeeds.

```
# 6 & 7. Success/Error Handling
if [ $? -eq 0 ]; then
    echo "Success: Backup created successfully at $ARCHIVE_PATH"
else
    echo "Error: Backup failed."
    exit 1
fi
```

7. Displays an appropriate error message if the backup fails.

Expected Outputs

```
Error: Usage: ./backup_check.sh <source_directory> <destination_directory>
[ec2-user@ip-172-31-31-162 ~]$
```

1. INVALID ARGUMENTS

```
[ec2-user@ip-172-31-31-162 ~]$ ./backup_check.sh /non/existent/path ~/test_backup
Error: Source directory '/non/existent/path' does not exist.
```

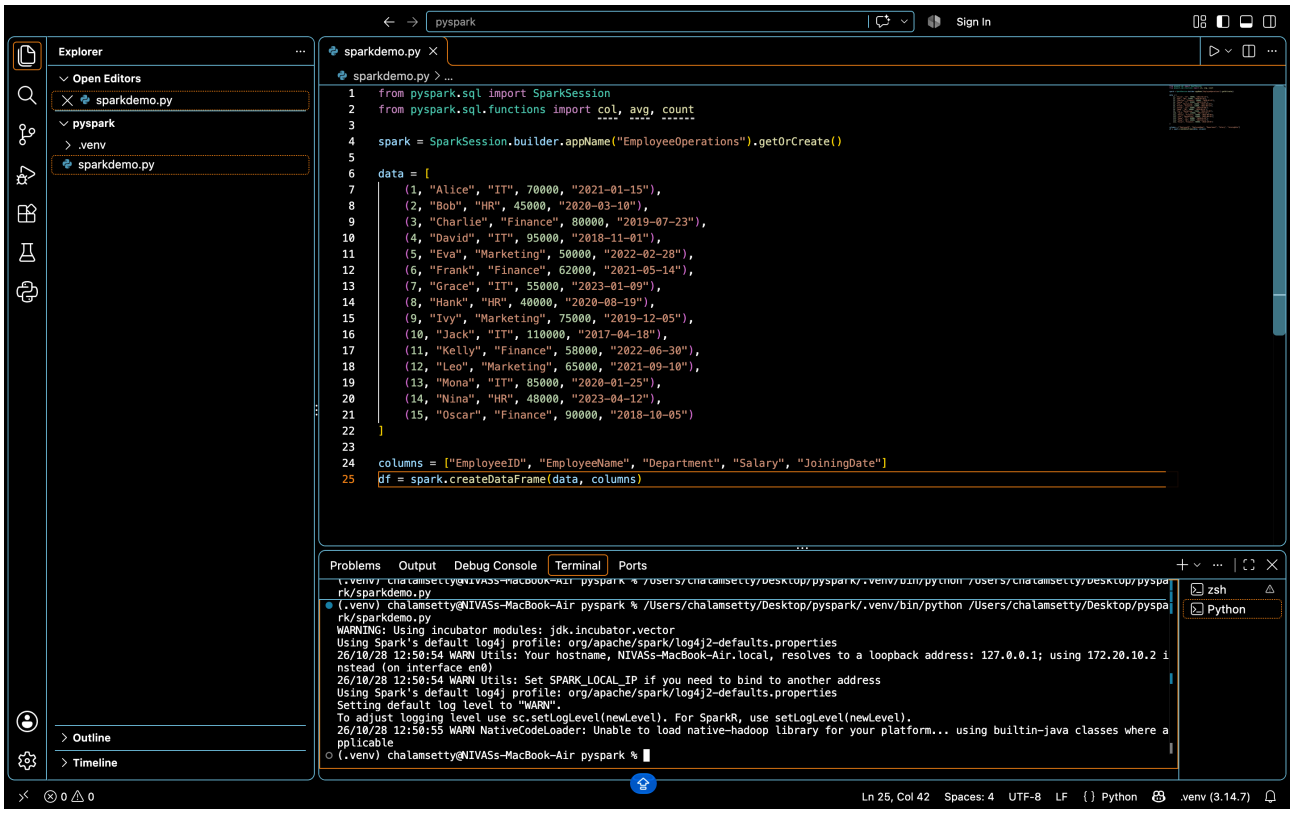
2. INVALID SOURCE DIRECTORY

3. SUCCESSFUL EXECUTION

```
Error: Source directory '/home/ec2-user/test_source' does not exist.
[ec2-user@ip-172-31-31-162 ~]$ mkdir ~/test_source
touch ~/test_source/file1.txt ~/test_source/file2.txt
[ec2-user@ip-172-31-31-162 ~]$ ./backup_check.sh ~/test_source ~/new_backup_folder
Notice: Destination directory '/home/ec2-user/new_backup_folder' does not exist. Creating it now...
Creating backup of '/home/ec2-user/test_source'...
Success: Backup created successfully at /home/ec2-user/new_backup_folder/backup_20261028_064348.tar.gz
[ec2-user@ip-172-31-31-162 ~]$
```

PySpark DataFrame Operations

Create a PySpark DataFrame



```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, avg, count

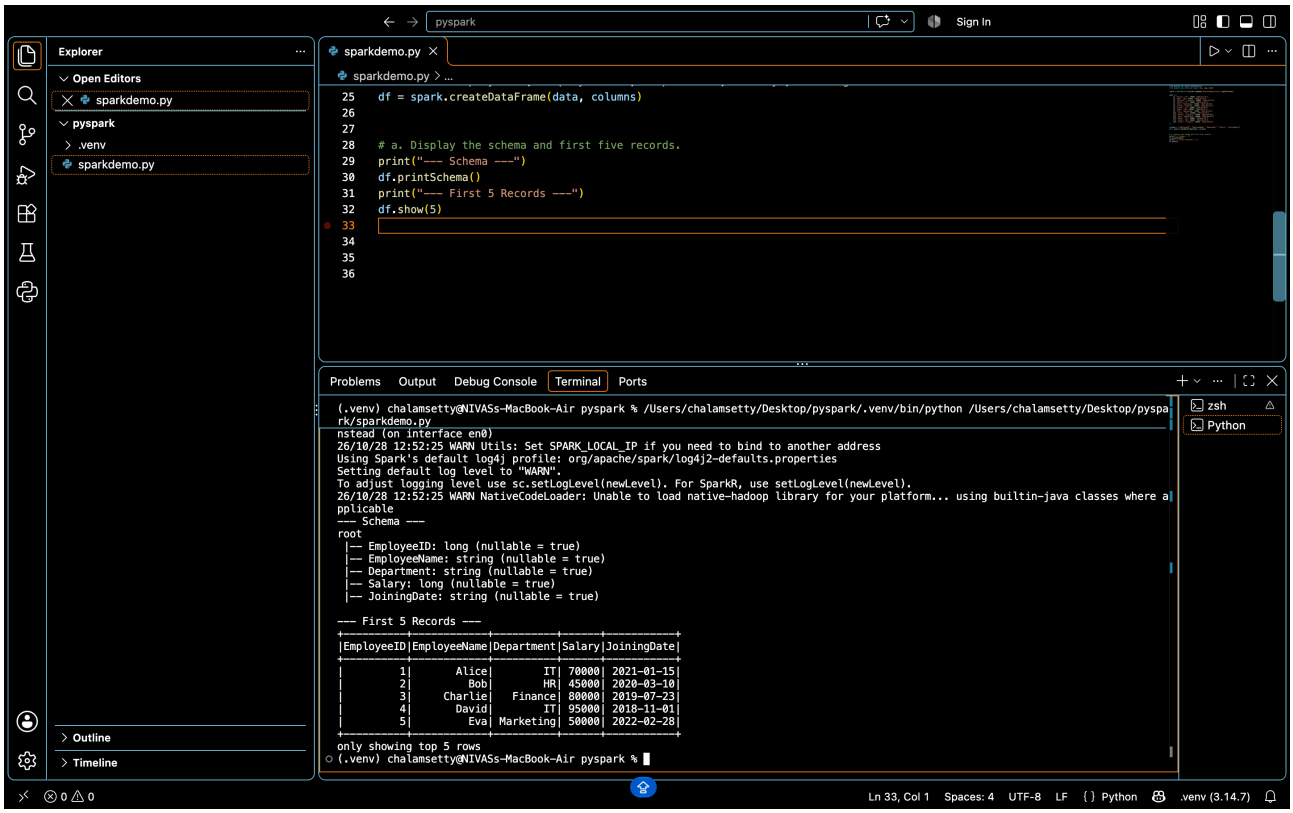
spark = SparkSession.builder.appName("EmployeeOperations").getOrCreate()

data = [
    (1, "Alice", "IT", 70000, "2021-01-15"),
    (2, "Bob", "HR", 45000, "2020-03-10"),
    (3, "Charlie", "Finance", 80000, "2019-07-23"),
    (4, "David", "IT", 95000, "2018-11-01"),
    (5, "Eva", "Marketing", 50000, "2022-02-28"),
    (6, "Frank", "Finance", 62000, "2021-05-14"),
    (7, "Grace", "IT", 55000, "2023-01-09"),
    (8, "Hank", "HR", 40000, "2020-08-19"),
    (9, "Ivy", "Marketing", 75000, "2019-12-05"),
    (10, "Jack", "IT", 110000, "2017-04-18"),
    (11, "Kelly", "Finance", 58000, "2022-06-30"),
    (12, "Leo", "Marketing", 65000, "2021-09-10"),
    (13, "Mona", "IT", 85000, "2020-01-25"),
    (14, "Nina", "HR", 48000, "2023-04-12"),
    (15, "Oscar", "Finance", 90000, "2018-10-05")
]

columns = ["EmployeeID", "EmployeeName", "Department", "Salary", "JoiningDate"]
df = spark.createDataFrame(data, columns)
```

```
(.venv) chalamsetty@NIVASs-MacBook-Air pyspark % /Users/chalamsetty/Desktop/pyspark/.venv/bin/python /Users/chalamsetty/Desktop/pyspark/sparkdemo.py
WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
26/10/28 12:50:54 WARN Utils: Your hostname, NIVASs-MacBook-Air.local, resolves to a loopback address: 127.0.0.1; using 172.20.10.2 instead (on interface en0)
26/10/28 12:50:54 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/10/28 12:50:55 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
(.venv) chalamsetty@NIVASs-MacBook-Air pyspark %
```

a. Display the schema and first five record

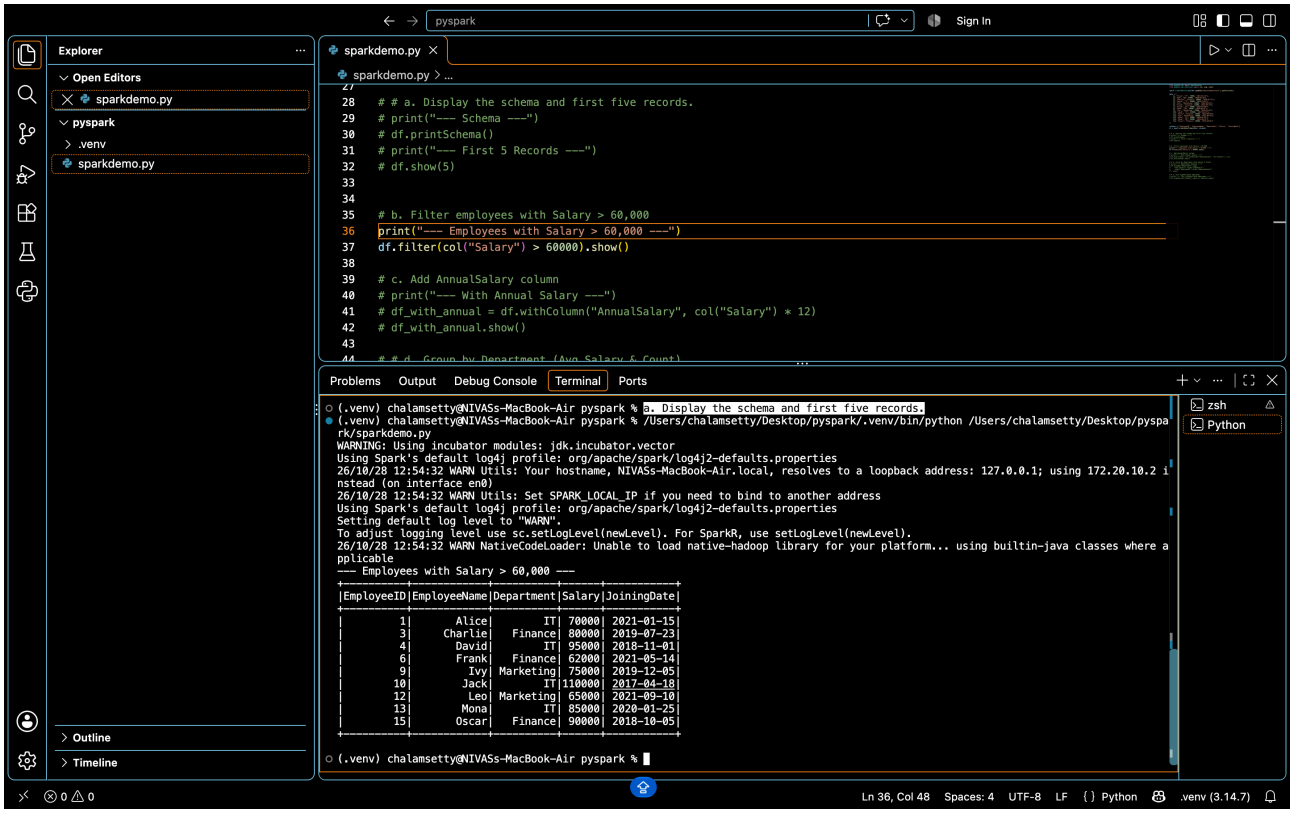


```
# a. Display the schema and first five records.
print("--- Schema ---")
df.printSchema()
print("--- First 5 Records ---")
df.show(5)
```

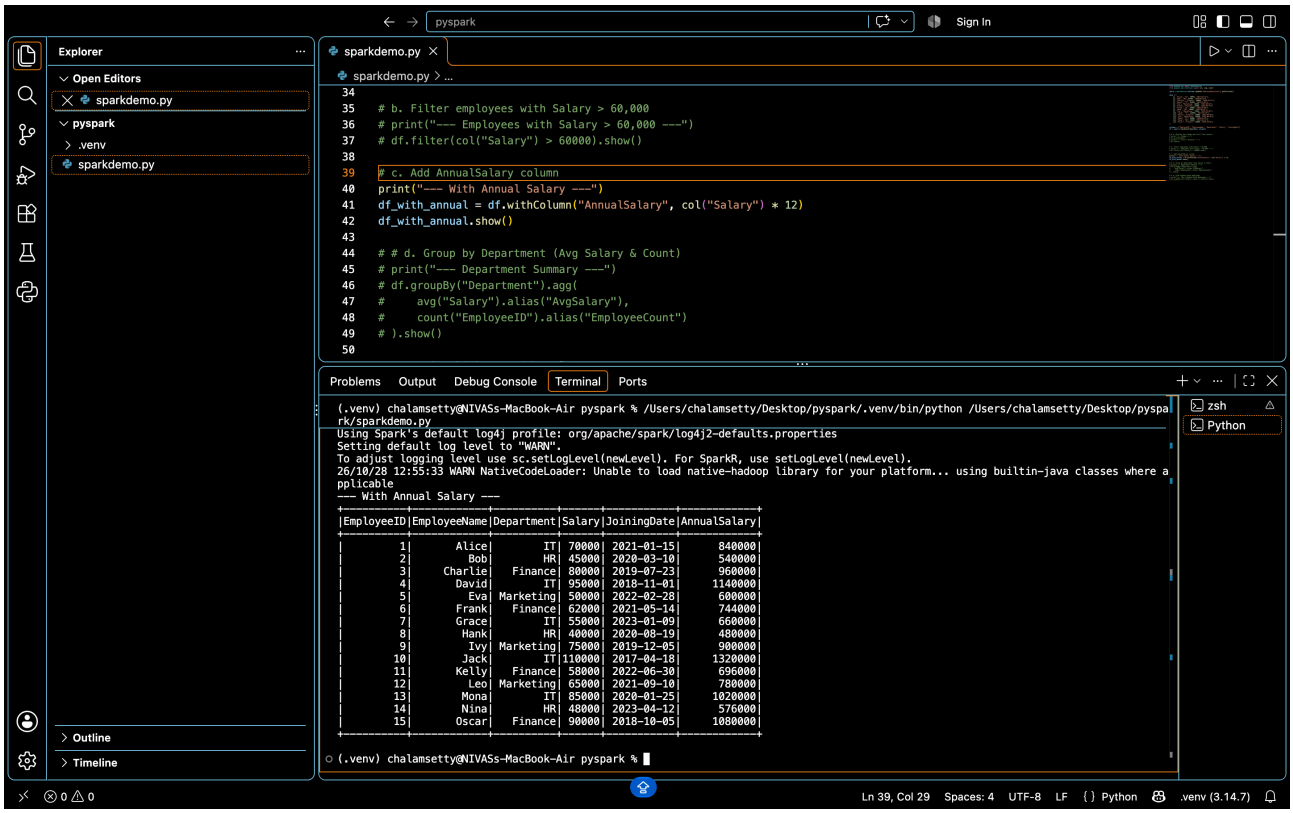
```
(.venv) chalamsetty@NIVASs-MacBook-Air pyspark % /Users/chalamsetty/Desktop/pyspark/.venv/bin/python /Users/chalamsetty/Desktop/pyspark/sparkdemo.py
nstead (on interface en0)
26/10/28 12:52:25 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/10/28 12:52:25 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
--- Schema ---
root
 |-- EmployeeID: long (nullable = true)
 |-- EmployeeName: string (nullable = true)
 |-- Department: string (nullable = true)
 |-- Salary: long (nullable = true)
 |-- JoiningDate: string (nullable = true)

--- First 5 Records ---
+----------+------------+----------+------+-----------+
|EmployeeID|EmployeeName|Department|Salary|JoiningDate|
+----------+------------+----------+------+-----------+
|         1|       Alice|        IT| 70000| 2021-01-15|
|         2|         Bob|        HR| 45000| 2020-03-10|
|         3|     Charlie|   Finance| 80000| 2019-07-23|
|         4|       David|        IT| 95000| 2018-11-01|
|         5|         Eva| Marketing| 50000| 2022-02-28|
+----------+------------+----------+------+-----------+
only showing top 5 rows
(.venv) chalamsetty@NIVASs-MacBook-Air pyspark %
```

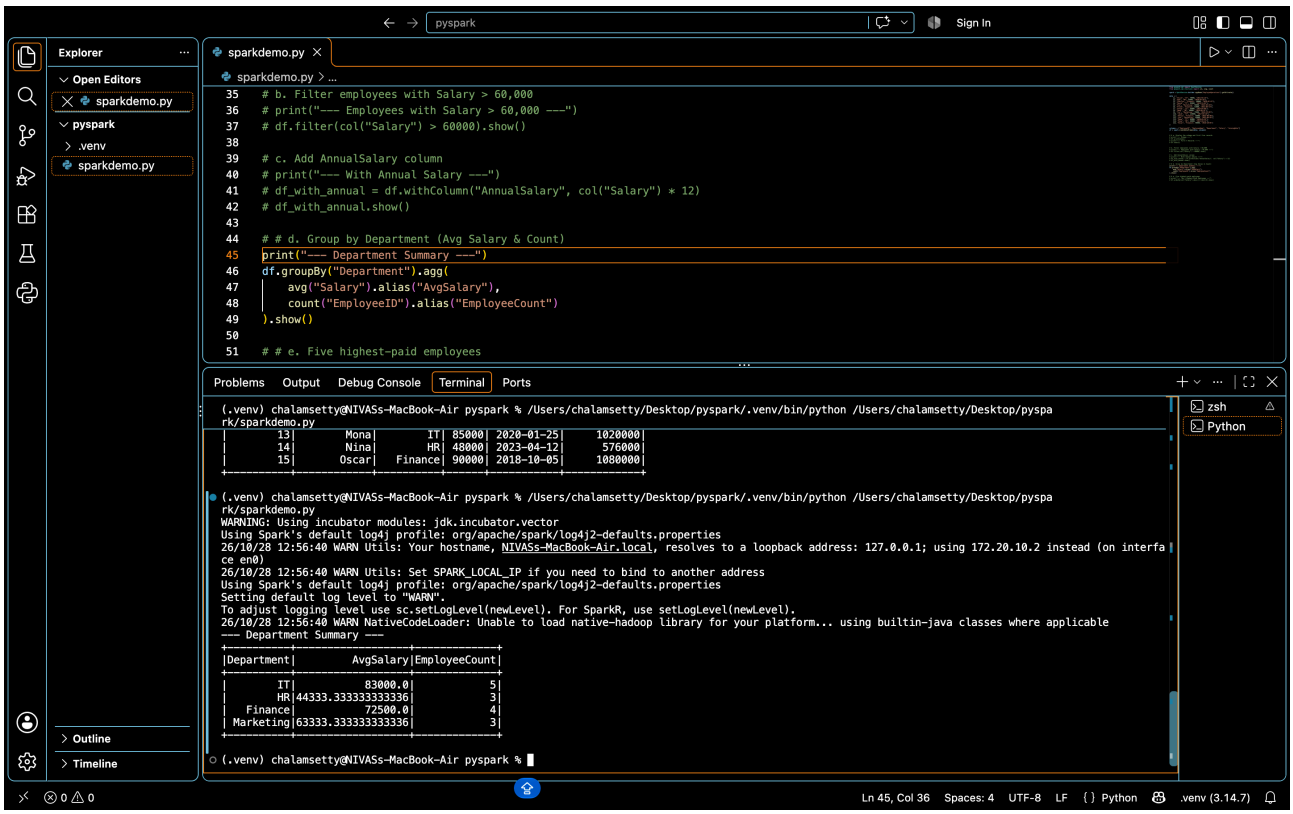
b. Filter employees whose monthly salary is greater than ₹60,000.



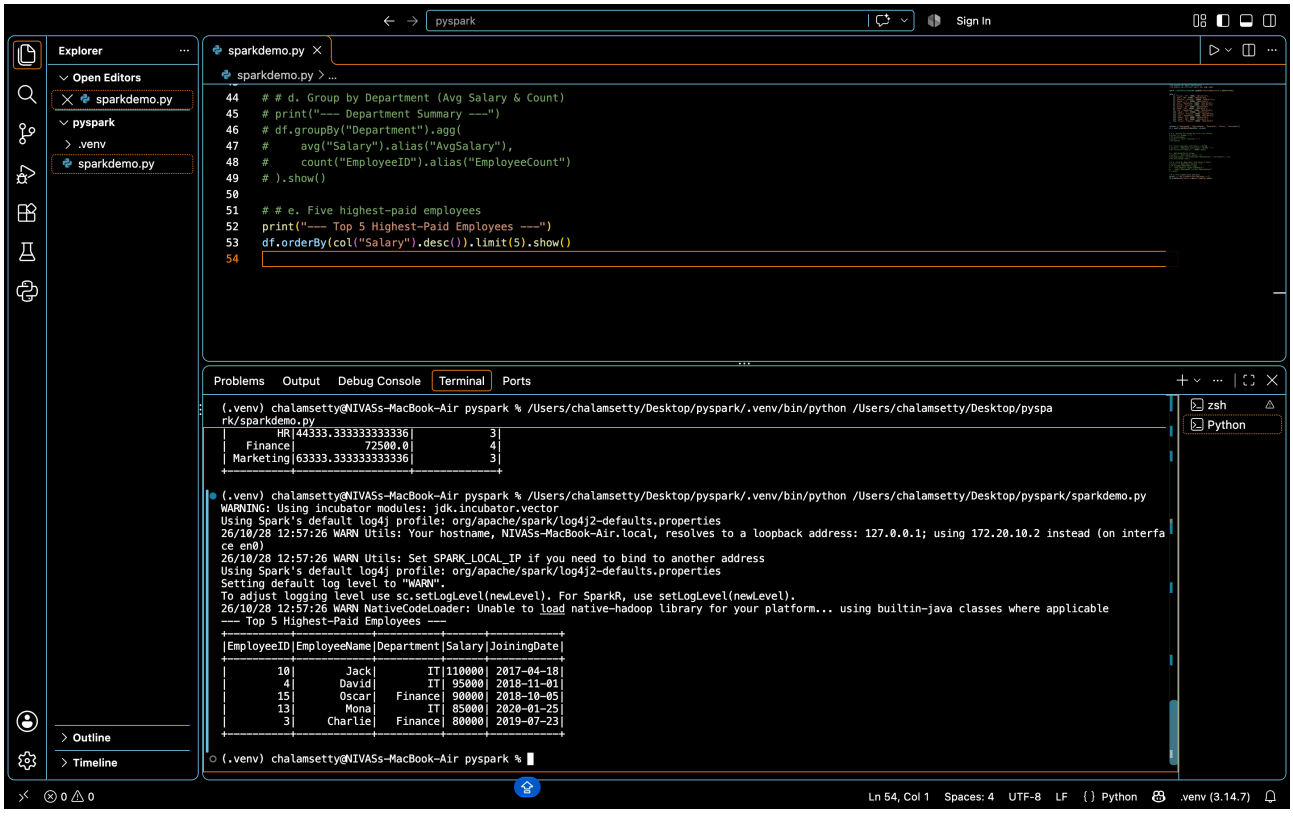
c. Add an AnnualSalary column calculated from the monthly salary



d. Group employees by department and calculate the average salary and employee count

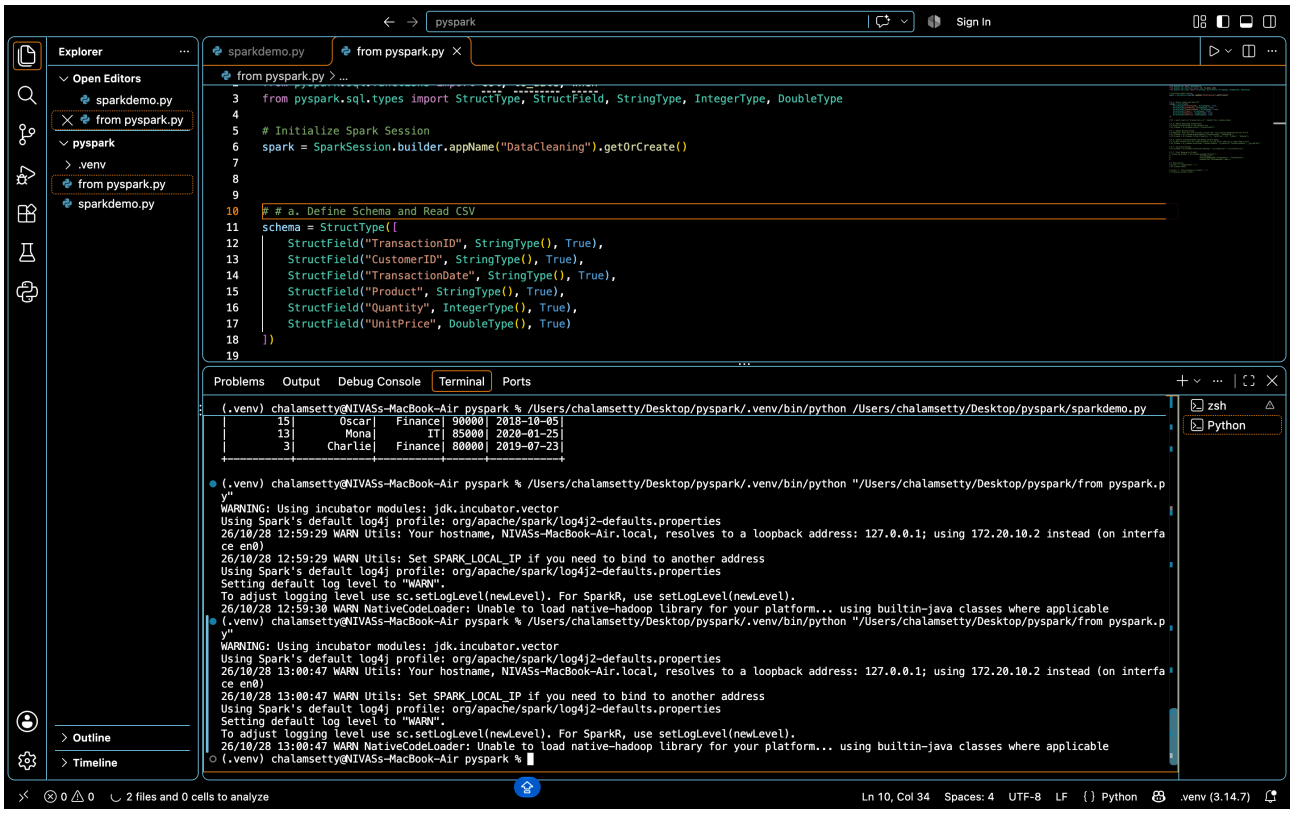


e. Display the five highest-paid employees

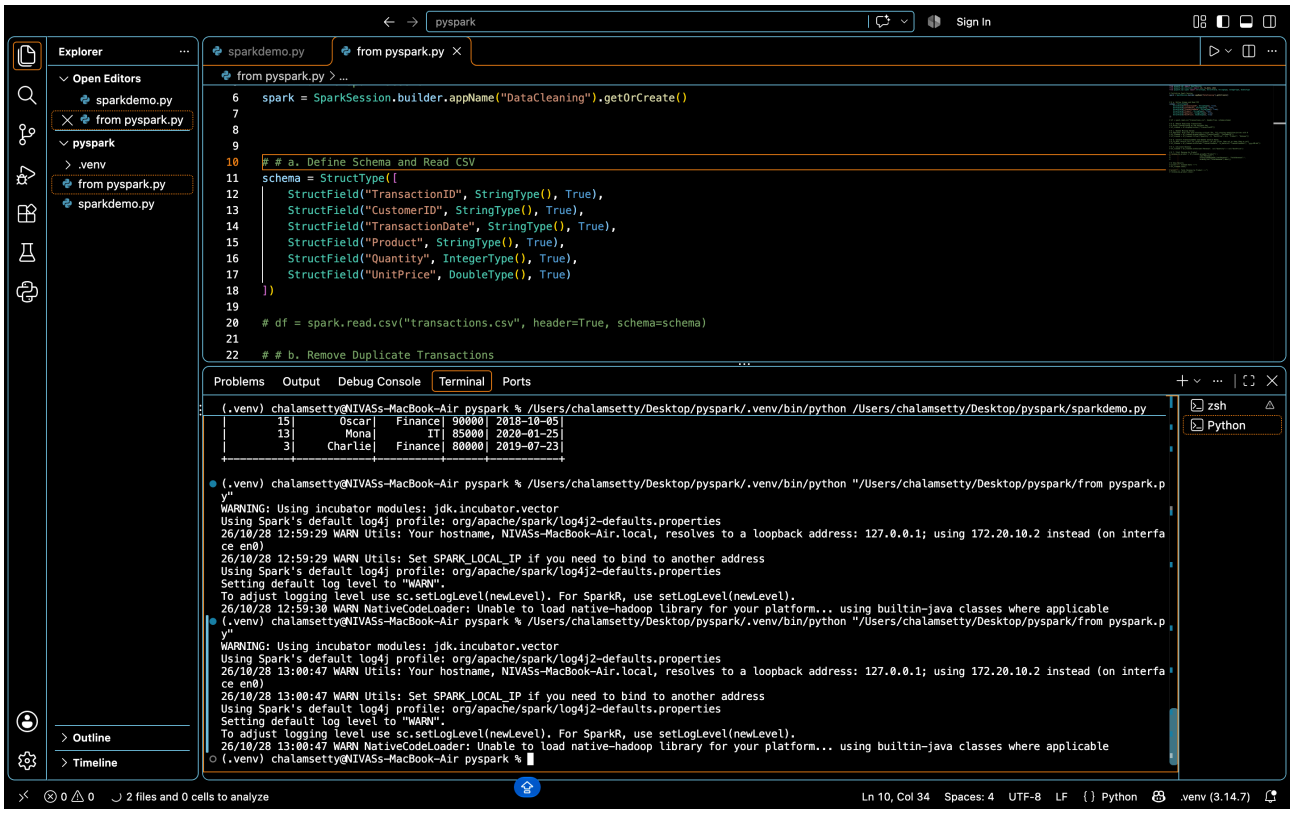


Data Cleaning and Aggregation

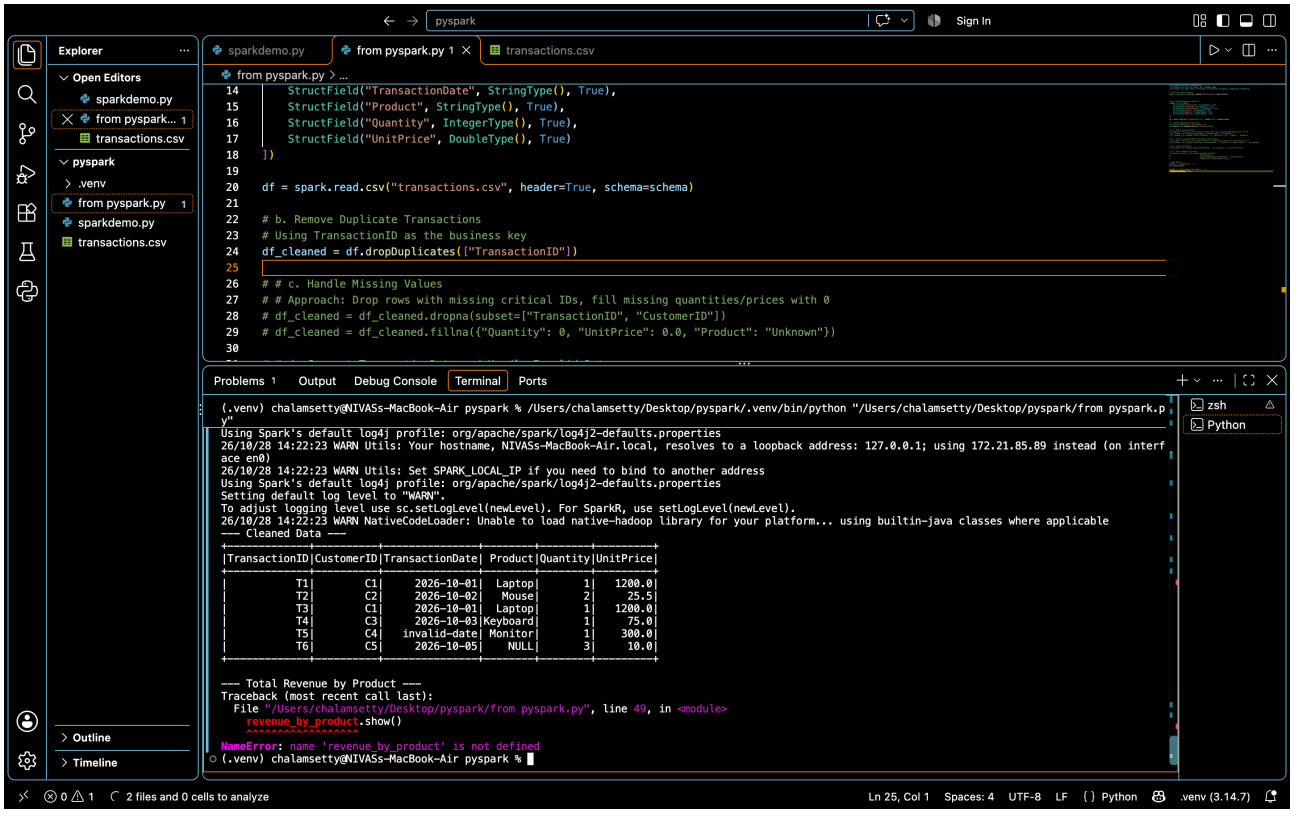
A company receives a CSV file named transactions.csv containing:



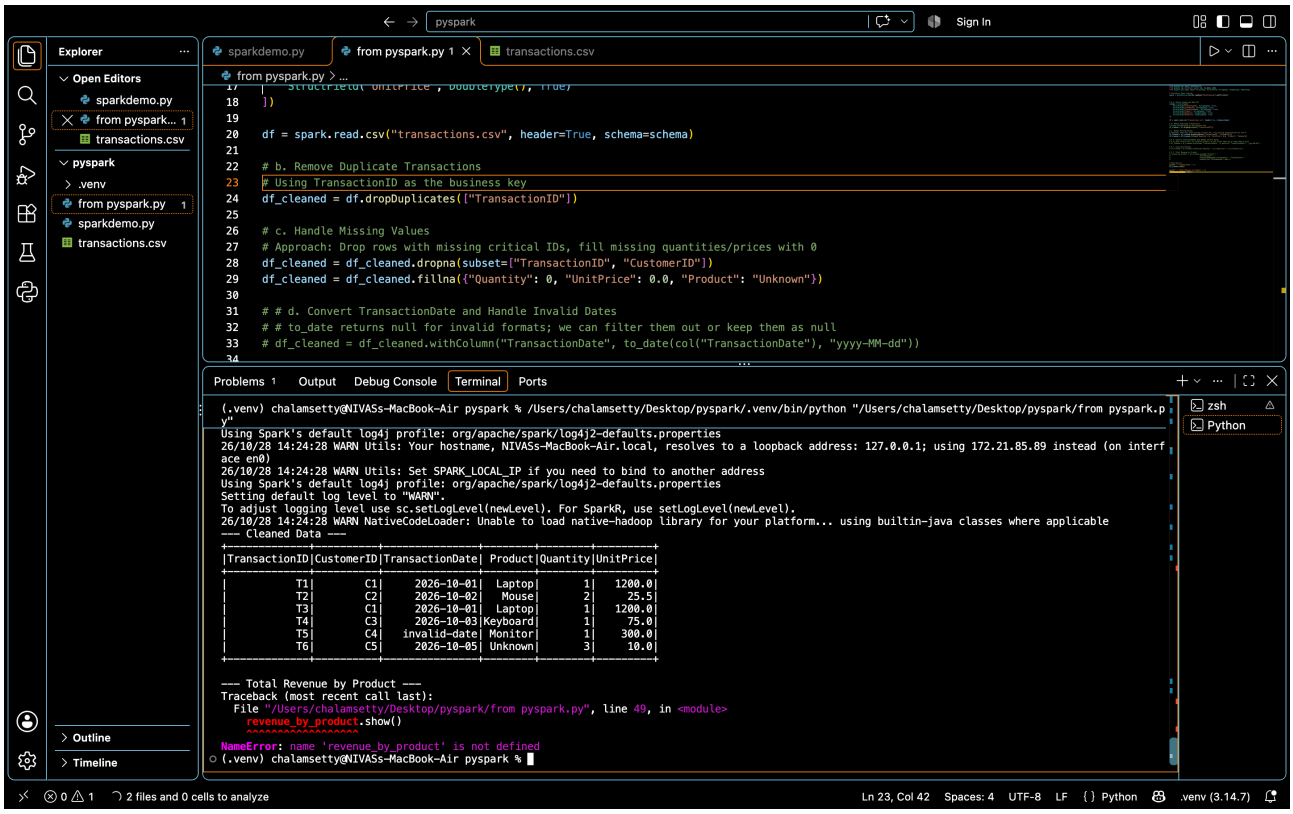
a. Read the CSV file using an appropriate schema and header configuration



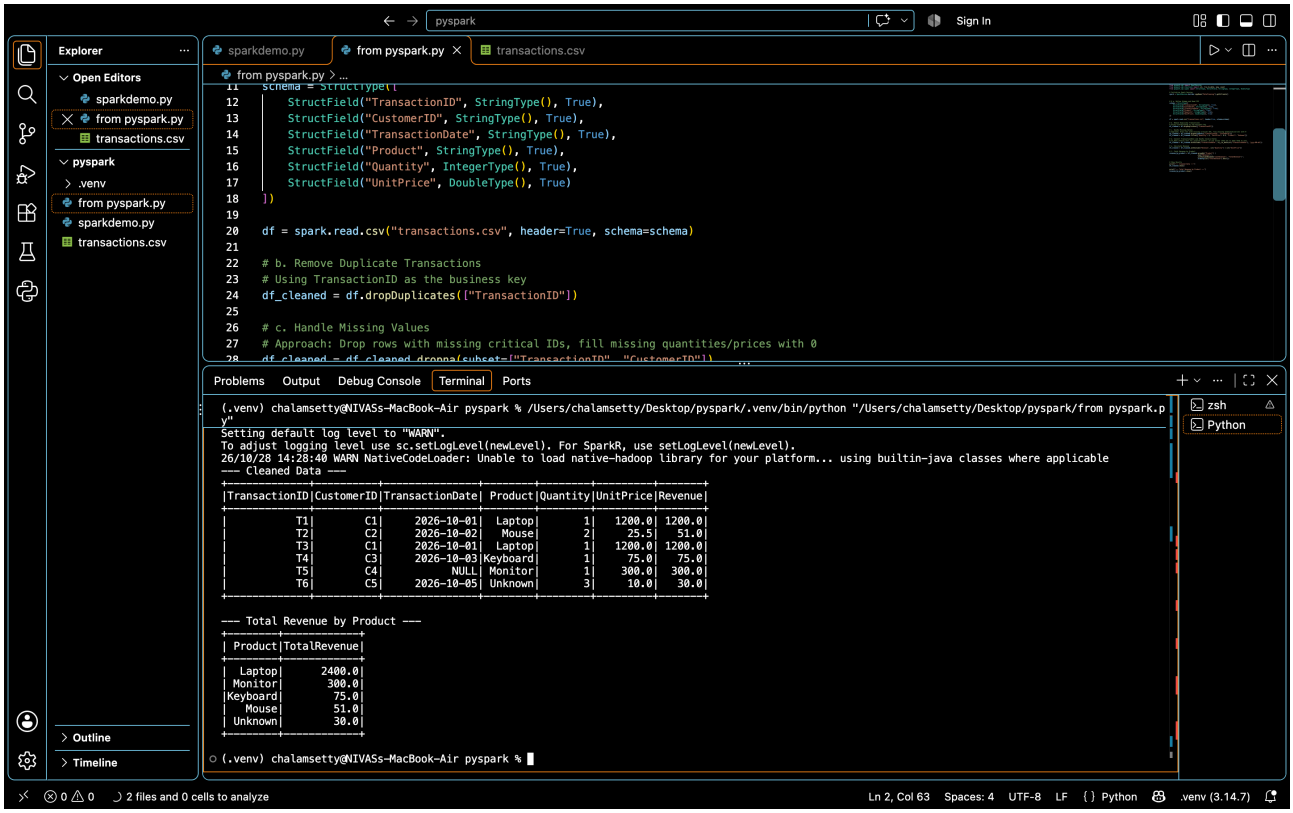
b. Identify and remove duplicate transactions using a suitable business key



c. Handle missing values in required fields and explain your chosen approach.



d. **Convert TransactionDate into a date type and handle invalid dates**



```
# b. Remove Duplicate Transactions
# Using TransactionID as the business key
df_cleaned = df.dropDuplicates(["TransactionID"])

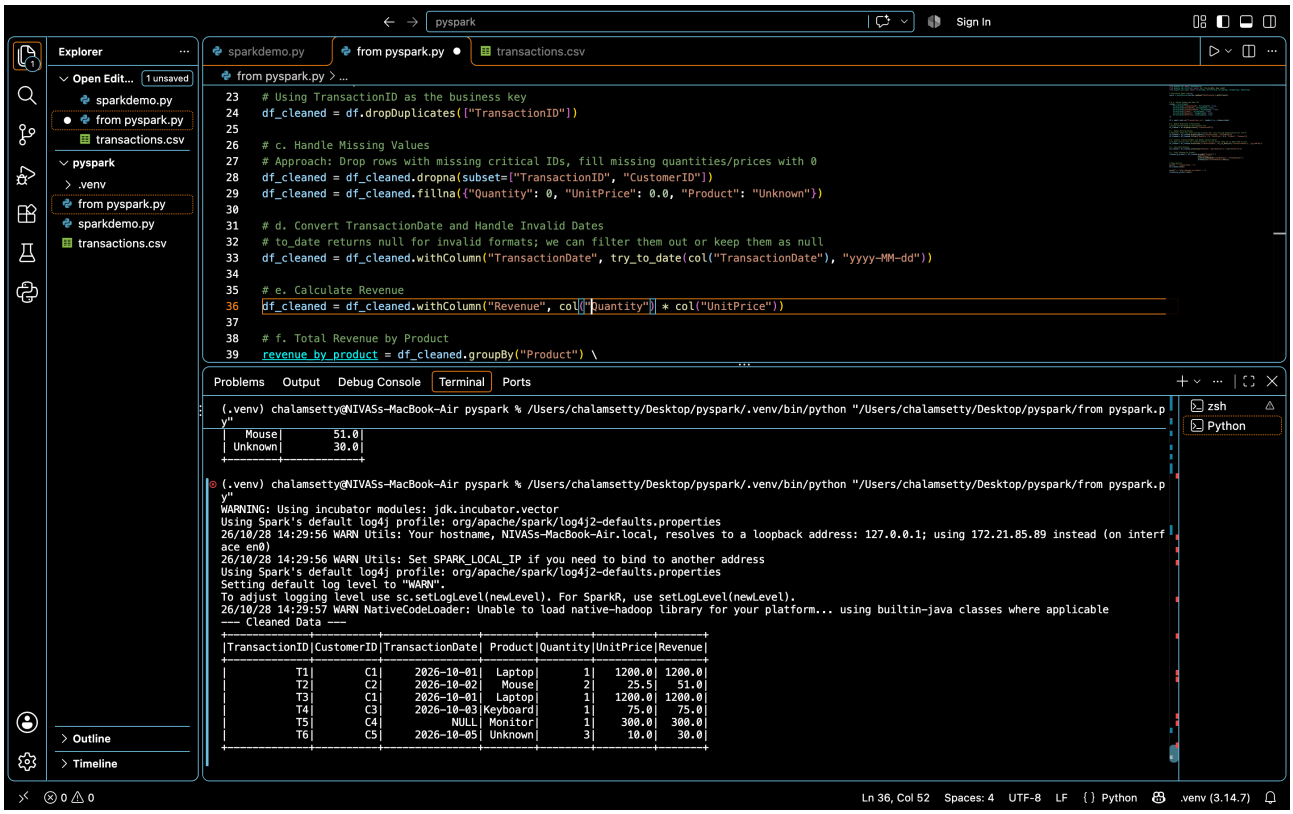
# c. Handle Missing Values
# Approach: Drop rows with missing critical IDs, fill missing quantities/prices with 0
```

```
(.venv) chalamsetty@NIVASs-MacBook-Air pyspark % /Users/chalamsetty/Desktop/pyspark/.venv/bin/python "/Users/chalamsetty/Desktop/pyspark/from pyspark.py"
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/10/28 14:28:40 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
--- Cleaned Data ---
+-------------+----------+---------------+-------+--------+---------+-------+
|TransactionID|CustomerID|TransactionDate|Product|Quantity|UnitPrice|Revenue|
+-------------+----------+---------------+-------+--------+---------+-------+
|           T1|        C1|     2026-10-01| Laptop|       1|   1200.0| 1200.0|
|           T2|        C2|     2026-10-02|  Mouse|       2|     25.5|   51.0|
|           T3|        C1|     2026-10-01| Laptop|       1|   1200.0| 1200.0|
|           T4|        C3|     2026-10-03|Keyboard|       1|     75.0|   75.0|
|           T5|        C4|           NULL|Monitor|       1|    300.0|  300.0|
|           T6|        C5|     2026-10-05|Unknown|       3|     10.0|   30.0|
+-------------+----------+---------------+-------+--------+---------+-------+

--- Total Revenue by Product ---
+--------+------------+
| Product|TotalRevenue|
+--------+------------+
|  Laptop|      2400.0|
| Monitor|       300.0|
|Keyboard|        75.0|
|   Mouse|        51.0|
| Unknown|        30.0|
+--------+------------+

(.venv) chalamsetty@NIVASs-MacBook-Air pyspark %
```

e. Calculate revenue using Quantity × UnitPrice.



```
# Using TransactionID as the business key
df_cleaned = df.dropDuplicates(["TransactionID"])

# c. Handle Missing Values
# Approach: Drop rows with missing critical IDs, fill missing quantities/prices with 0
df_cleaned = df_cleaned.dropna(subset=["TransactionID", "CustomerID"])
df_cleaned = df_cleaned.fillna({"Quantity": 0, "UnitPrice": 0.0, "Product": "Unknown"})

# d. Convert TransactionDate and Handle Invalid Dates
# to_date returns null for invalid formats; we can filter them out or keep them as null
df_cleaned = df_cleaned.withColumn("TransactionDate", try_to_date(col("TransactionDate"), "yyyy-MM-dd"))

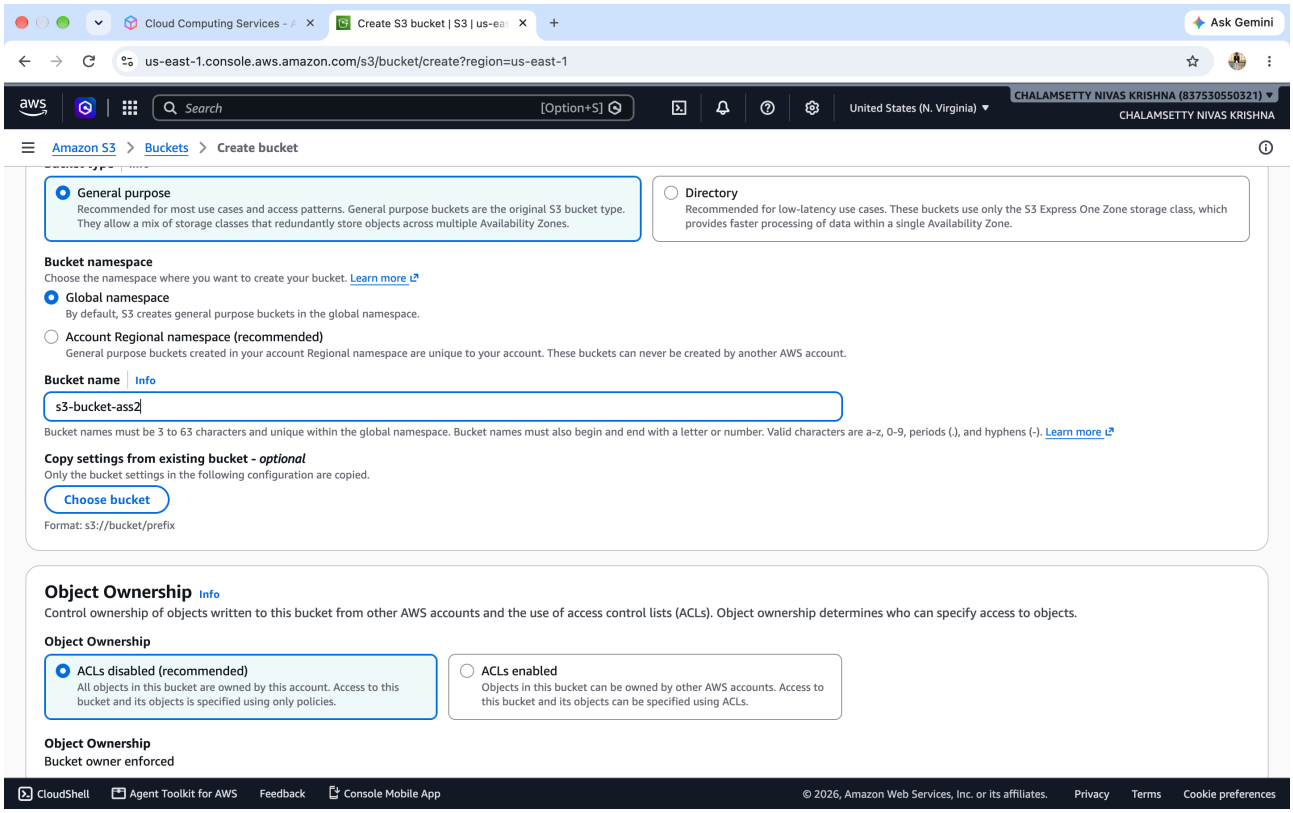
# e. Calculate Revenue
df_cleaned = df_cleaned.withColumn("Revenue", col("Quantity") * col("UnitPrice"))

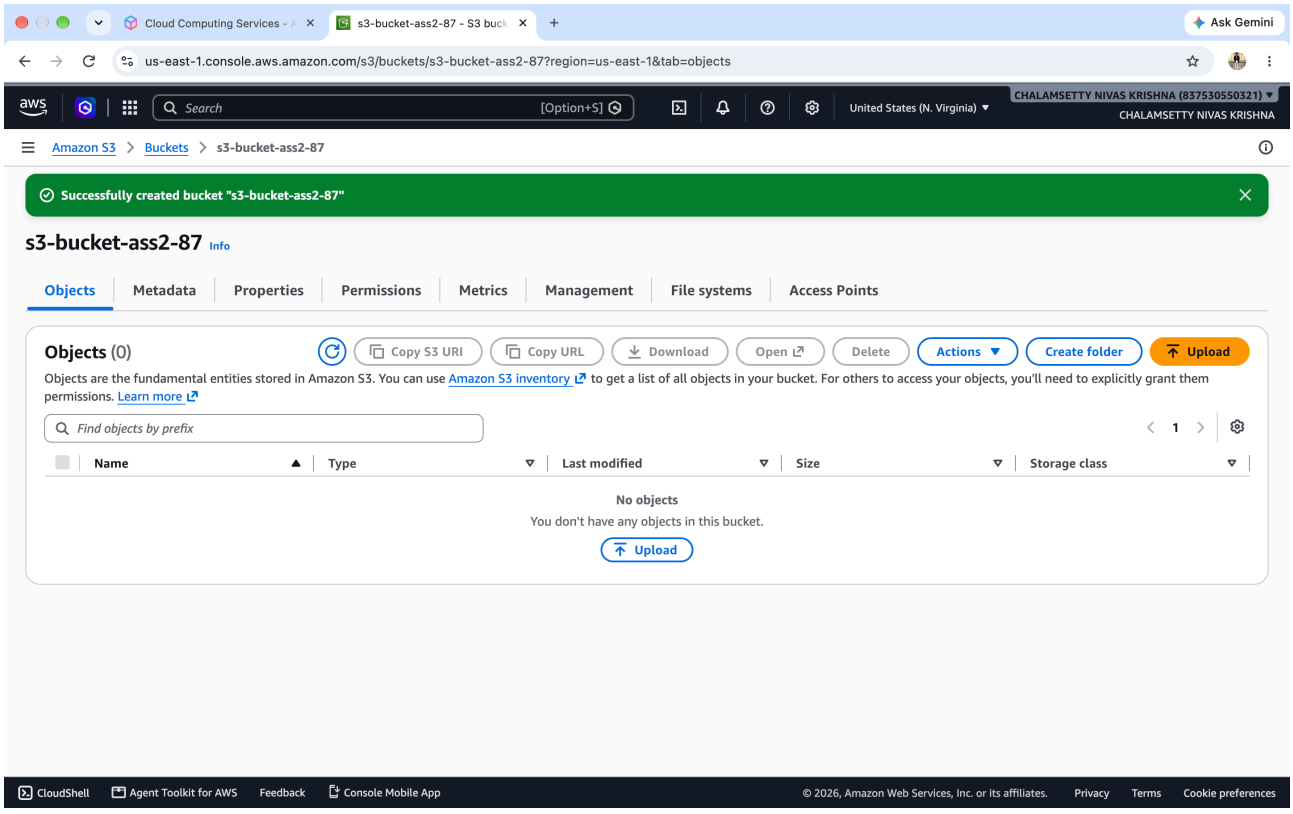
# f. Total Revenue by Product
revenue_by_product = df_cleaned.groupBy("Product") \
```

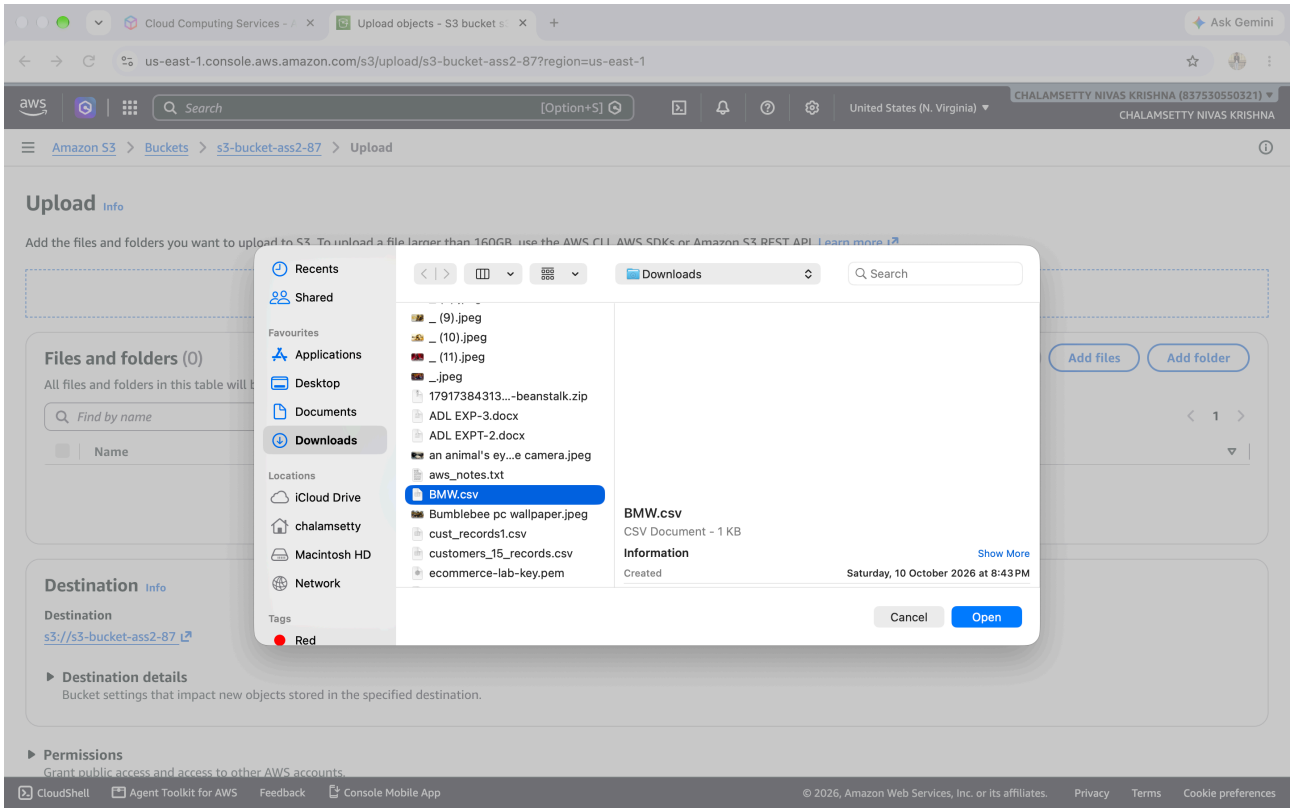
```
(.venv) chalamsetty@NIVASs-MacBook-Air pyspark % /Users/chalamsetty/Desktop/pyspark/.venv/bin/python "/Users/chalamsetty/Desktop/pyspark/from pyspark.py"
|   Mouse|        51.0|
| Unknown|        30.0|
+--------+------------+

(.venv) chalamsetty@NIVASs-MacBook-Air pyspark % /Users/chalamsetty/Desktop/pyspark/.venv/bin/python "/Users/chalamsetty/Desktop/pyspark/from pyspark.py"
WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
26/10/28 14:29:56 WARN Utils: Your hostname, NIVASs-MacBook-Air.local, resolves to a loopback address: 127.0.0.1; using 172.21.85.89 instead (on interface en0)
26/10/28 14:29:56 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/10/28 14:29:57 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
--- Cleaned Data ---
+-------------+----------+---------------+-------+--------+---------+-------+
|TransactionID|CustomerID|TransactionDate|Product|Quantity|UnitPrice|Revenue|
+-------------+----------+---------------+-------+--------+---------+-------+
|           T1|        C1|     2026-10-01| Laptop|       1|   1200.0| 1200.0|
|           T2|        C2|     2026-10-02|  Mouse|       2|     25.5|   51.0|
|           T3|        C1|     2026-10-01| Laptop|       1|   1200.0| 1200.0|
|           T4|        C3|     2026-10-03|Keyboard|       1|     75.0|   75.0|
|           T5|        C4|           NULL|Monitor|       1|    300.0|  300.0|
|           T6|        C5|     2026-10-05|Unknown|       3|     10.0|   30.0|
+-------------+----------+---------------+-------+--------+---------+-------+
```

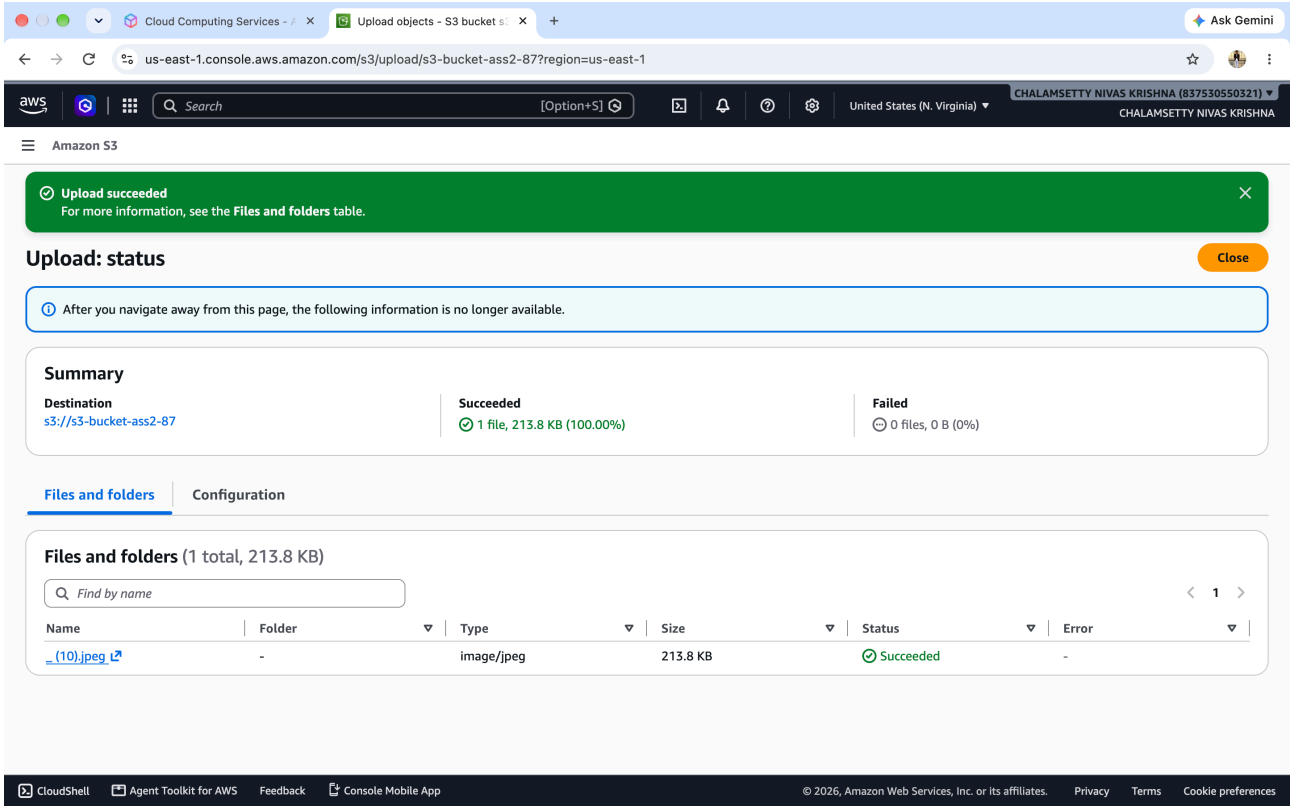
S3 and IAM Security

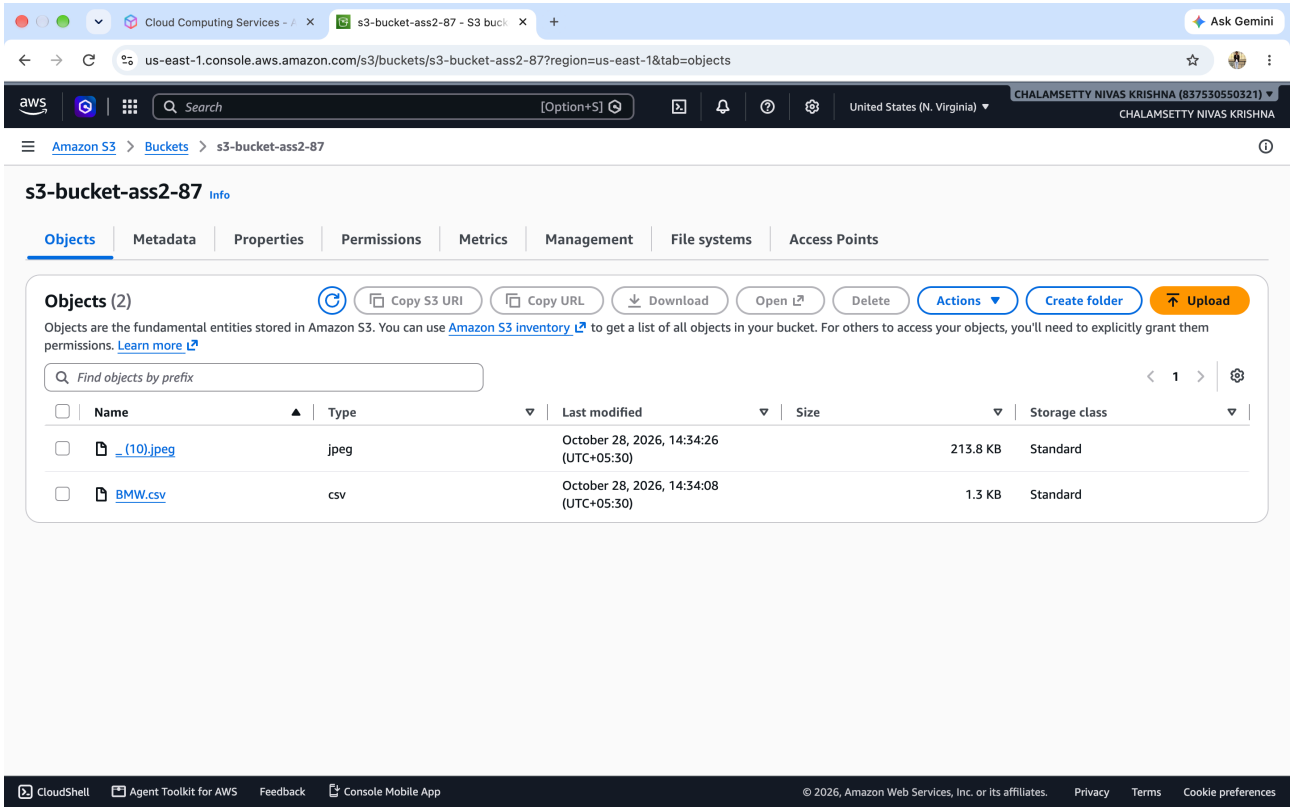






a. **Create an S3 bucket with Block Public Access enabled and upload a sample file.**





b. Design an IAM policy that grants read access to objects in the designated bucket without granting access to unrelated buckets.

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
                "s3:GetObject"
            ],
            "Resource": "arn:aws:s3:::hr-confidential-reports/*"
        },
        {
            "Effect": "Allow",
            "Action": [
                "s3:ListBucket"
            ],
            "Resource": "arn:aws:s3:::hr-confidential-reports"
        }
    ]
}
```

c. Explain how you would verify that an unauthorized user cannot read the file.

To verify that an unauthorized user cannot read the file:

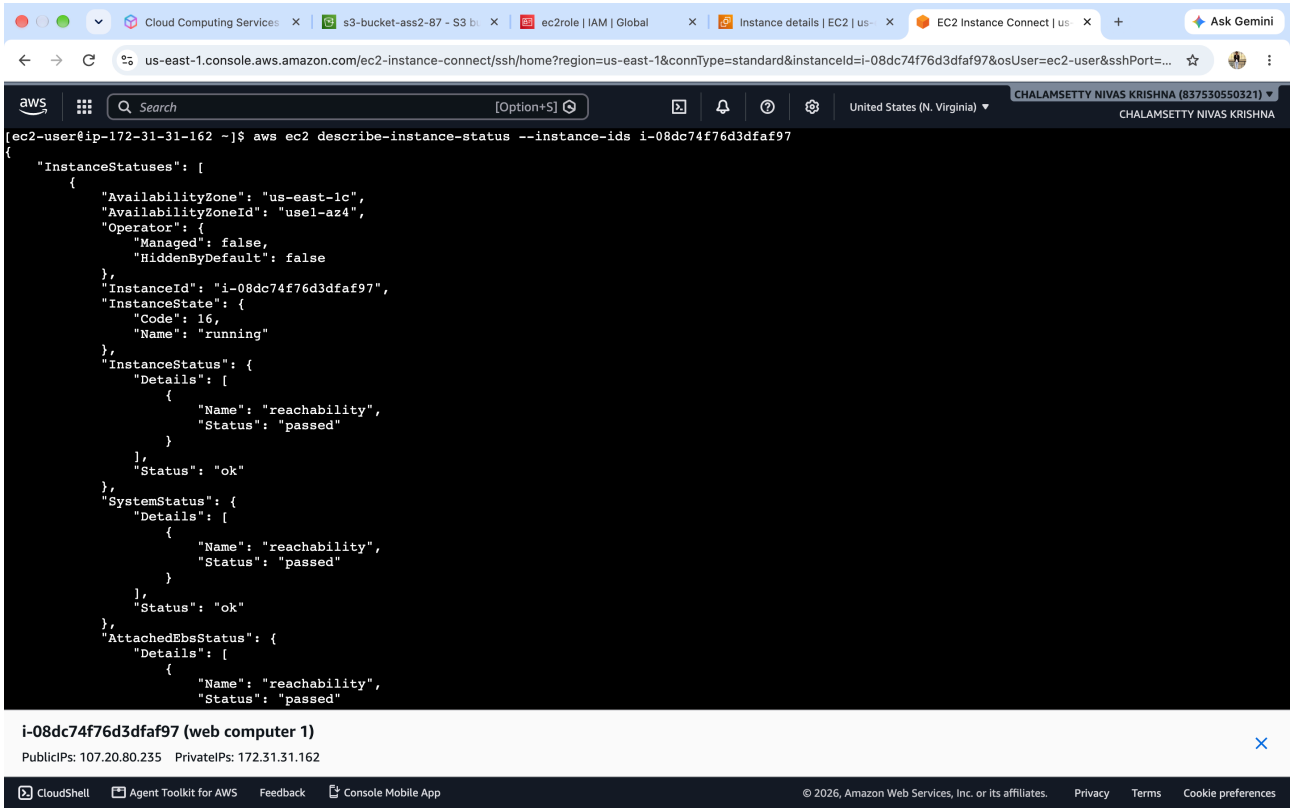
Attempt Public Access: Try to open the Object URL in an incognito browser window. It should return an AccessDenied error.

Cross-Account/User Test: Log in as an IAM user who does *not* have the HR policy attached. Attempt to run aws s3 cp s3://hr-confidential-reports/sample_report.pdf . via the CLI. This should also return an AccessDenied error.

d. Explain how encryption at rest protects the stored data.

Encryption at rest (such as SSE-S3 or SSE-KMS) protects data by automatically encrypting the files before they are written to the physical disks in the AWS data center, and decrypting them only when they are accessed by an authorized user.

EC2 Troubleshooting and CloudWatch



a. **Check the EC2 instance state and status checks.**

b. Explain how security groups and route tables can affect application connectivity.

ANS :

Security Groups: These control inbound and outbound traffic at the instance level. If your security group does not explicitly allow inbound traffic on your application's port (e.g., port 80 for HTTP or 443 for HTTPS), users will be unable to connect.

Route Tables: These determine where network traffic is directed within your VPC. If your subnet's route table does not have a route (0.0.0.0/0) pointing to an Internet Gateway, your instance will be isolated from the internet.

c. Explain how to configure a CloudWatch alarm when CPU utilization exceeds 85% for a defined evaluation period.

To set up an alarm for high CPU usage:

Open the CloudWatch Console and select Alarms > Create Alarm.

Choose the **EC2** metric for **CPUUtilization** for your specific instance.

Set the threshold condition to **Static**, **Greater**, and enter **85**.

Event-Driven Processing

a. Identify a suitable service for executing the processing code

ANS :
AWS Lambda is ideal for this use case as it scales automatically, requires no server management, and only charges for the compute time used.

b. Explain how an S3 upload can trigger processing.

ANS:
S3 Event Notifications can be configured on the bucket to send an event to Lambda whenever an ObjectCreated event occurs (e.g., a file is uploaded).

c. Explain the IAM permissions needed to read the input file and write the results.

ANS :
The Lambda execution role needs s3:GetObject for the source bucket and s3:PutObject for the destination bucket.

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": ["s3:GetObject"],
            "Resource": "arn:aws:s3:::source-csv-bucket/*"
        },
        {
            "Effect": "Allow",
            "Action": ["s3:PutObject"],
            "Resource": "arn:aws:s3:::destination-processed-bucket/*"
        },
        {
            "Effect": "Allow",
            "Action": ["logs:CreateLogGroup", "logs:CreateLogStream", "lo
            "Resource": "arn:aws:logs:*:*:*"
        }
    ]
}
```

This policy grants the necessary permissions for the Lambda function to read the input, write the output, and write logs to CloudWatch.

d. Explain how errors and processing logs can be captured.
ANS :
Amazon CloudWatch Logs automatically captures standard output and errors from the Lambda function.

e. Explain how to prevent duplicate processing or recursive triggering.
ANS :
Write the processed output to a **different S3 bucket** (or a different folder prefix) than the input bucket to prevent the output from triggering the Lambda function again.

State Machine Design

To coordinate the sales file processing workflow, you can design an AWS Step Functions state machine using Task states for validation, transformation, storage, and notification.

```JSON
{
  "StartAt": "ValidateFile",
  "States": {
    "ValidateFile": {
      "Type": "Task",
      "Resource": "arn:aws:states:::lambda:invoke",
      "Next": "CheckValidation"
    },
    "CheckValidation": {
      "Type": "Choice",
      "Choices": [
        {
          "Variable": "$.Payload.isValid",
          "BooleanEquals": true,
          "Next": "TransformData"
        }
      ],
      "Default": "ValidationFailed"
    },
    "TransformData": {
      "Type": "Task",
      "Resource": "arn:aws:states:::lambda:invoke",
      "Next": "CalculateSales"
    },
    "CalculateSales": {
      "Type": "Task",
      "Resource": "arn:aws:states:::lambda:invoke",
      "Next": "StoreResults"
    },
    "StoreResults": {
      "Type": "Task",
      "Resource": "arn:aws:states:::aws-sdk:s3:putObject",
      "Next": "SendNotification"
    },
    "SendNotification": {
      "Type": "Task",
      "Resource": "arn:aws:states:::sns:publish",
      "End": true
    },
    "ValidationFailed": {
      "Type": "Fail",
      "Error": "InvalidFile",
      "Cause": "The input file failed validation checks."
    }
  }
}
```

a. Identify suitable states for validation, processing, storage, and notification.

ANS :

Use **Task** states for invoking Lambda (Validation, Transform, Calculate), an **S3 integration** for storage, and an **SNS integration** for the notification.

b. Explain how a Choice state can route invalid files to a failure path and valid files to processing.
ANS :
The **Execution History** in the AWS Console shows the exact input, output, and error stack trace for every step, making it easy to pinpoint where a failure occurred.

c. Explain how Retry and Catch can handle transient failures and permanent errors.
ANS :
The Choice state checks a variable (e.g., $.isValid) from the validation step. If true, it routes to "TransformData"; if false, it routes to a "ValidationFailed" **Fail** state.

d. Explain how the state machine can invoke Lambda functions or other supported AWS services.
ANS :
Step Functions invokes services using ARNs in the Resource field of a Task state (e.g., arn:aws:states:::lambda:invoke).

e. Explain how execution history helps troubleshoot a failed workflow.
ANS :
Retry automatically re-runs a state on transient errors (e.g., timeouts) with backoff. **Catch** intercepts permanent errors and routes the workflow to a fallback path (like an alert).

Step Functions Implementation

Here is the Amazon States Language (ASL) JSON definition for your sequential workflow. It includes the three required tasks, a retry policy for the transformation step, and a catch block for the loading step.

```
{
  "Comment": "Sequential Data Processing Workflow",
  "StartAt": "ValidateFile",
  "States": {
    "ValidateFile": {
      "Type": "Task",
      "Resource": "arn:aws:lambda:region:account:function:ValidateFile",
      "Next": "TransformData"
    },
    "TransformData": {
      "Type": "Task",
      "Resource": "arn:aws:lambda:region:account:function:TransformData",
      "Retry": [
        {
          "ErrorEquals": ["States.TaskFailed", "States.Timeout"],
          "IntervalSeconds": 2,
          "MaxAttempts": 3,
          "BackoffRate": 2.0
        }
      ],
      "Next": "LoadResults"
    },
    "LoadResults": {
      "Type": "Task",
      "Resource": "arn:aws:lambda:region:account:function:LoadResults",
      "Catch": [
        {
          "ErrorEquals": ["States.ALL"],
          "Next": "HandleFailure"
        }
      ],
      "End": true
    },
    "HandleFailure": {
      "Type": "Fail",
      "Error": "LoadError",
      "Cause": "The workflow failed during the LoadResults stage."
    }
  }
}
```

a. Write an ASL JSON definition containing the three sequential tasks. Use placeholders for resource ARNs where necessary.

b. Configure a retry policy for the transformation task with an appropriate error category, retry interval, and maximum attempts.

ANS :

```
"Comment": "Sequential Data Processing Workflow",
"StartAt": "ValidateFile",
"States": {
  "ValidateFile": {
    "Type": "Task",
    "Resource": "arn:aws:lambda:region:account:function:ValidateFile",
    "Next": "TransformData"
  },
  "TransformData": {
    "Type": "Task",
    "Resource": "arn:aws:lambda:region:account:function:TransformData",
    "Retry": [
      {
        "ErrorEquals": ["States.TaskFailed", "States.Timeout"],
        "IntervalSeconds": 2,
        "MaxAttempts": 3,
        "BackoffRate": 2.0
      }
    ],
    "Next": "LoadResults"
  },
  "LoadResults": {
    "Type": "Task",
    "Resource": "arn:aws:lambda:region:account:function:LoadResults",
    "Catch": [
      {
        "ErrorEquals": ["States.ALL"],
        "Next": "HandleFailure"
      }
```

c. Configure a Catch transition for the loading task that routes to an error-handling state.

ANS :

```
  },
  "HandleFailure": {
    "Type": "Fail",
    "Error": "LoadError",
    "Cause": "The workflow failed during the LoadResults stage."
  }
}
```

d. Explain how data from one task is passed to the next task.
ANS :
Data Passing: Step Functions passes data as a JSON payload. The output of ValidateFile automatically becomes the input for TransformData, unless modified using ResultPath or OutputPath.

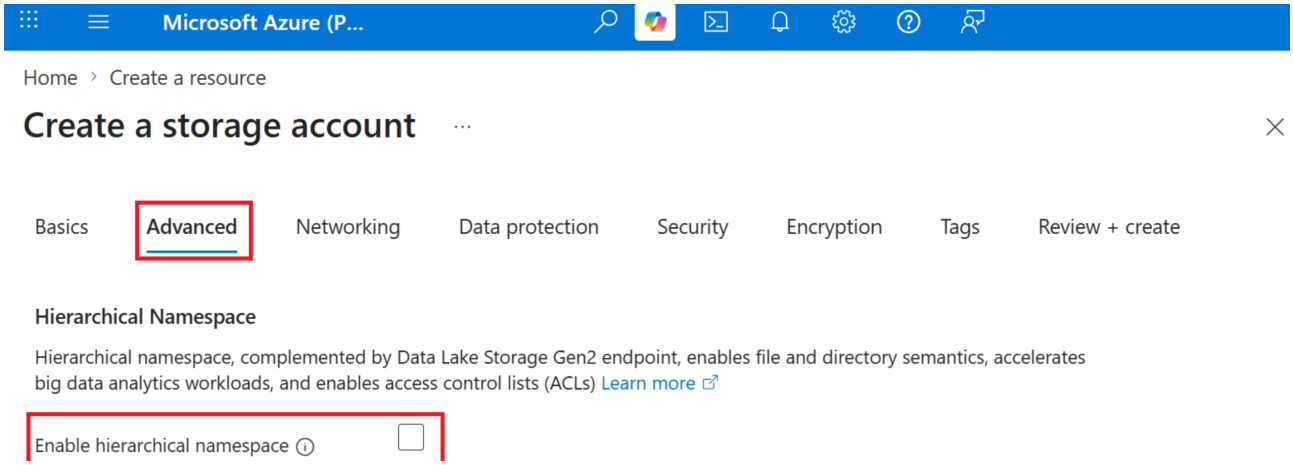
e. Explain how you would test successful and failed executions.
ANS :
Successful Execution: Start an execution with a valid mock JSON input and verify that all states turn green in the AWS Console.
Failed Execution: Pass an invalid input (or configure a mock Lambda to throw an error) to verify that the Retry logic triggers and that the Catch block successfully routes to the HandleFailure state.

Azure Data Lake Storage Gen2 Security

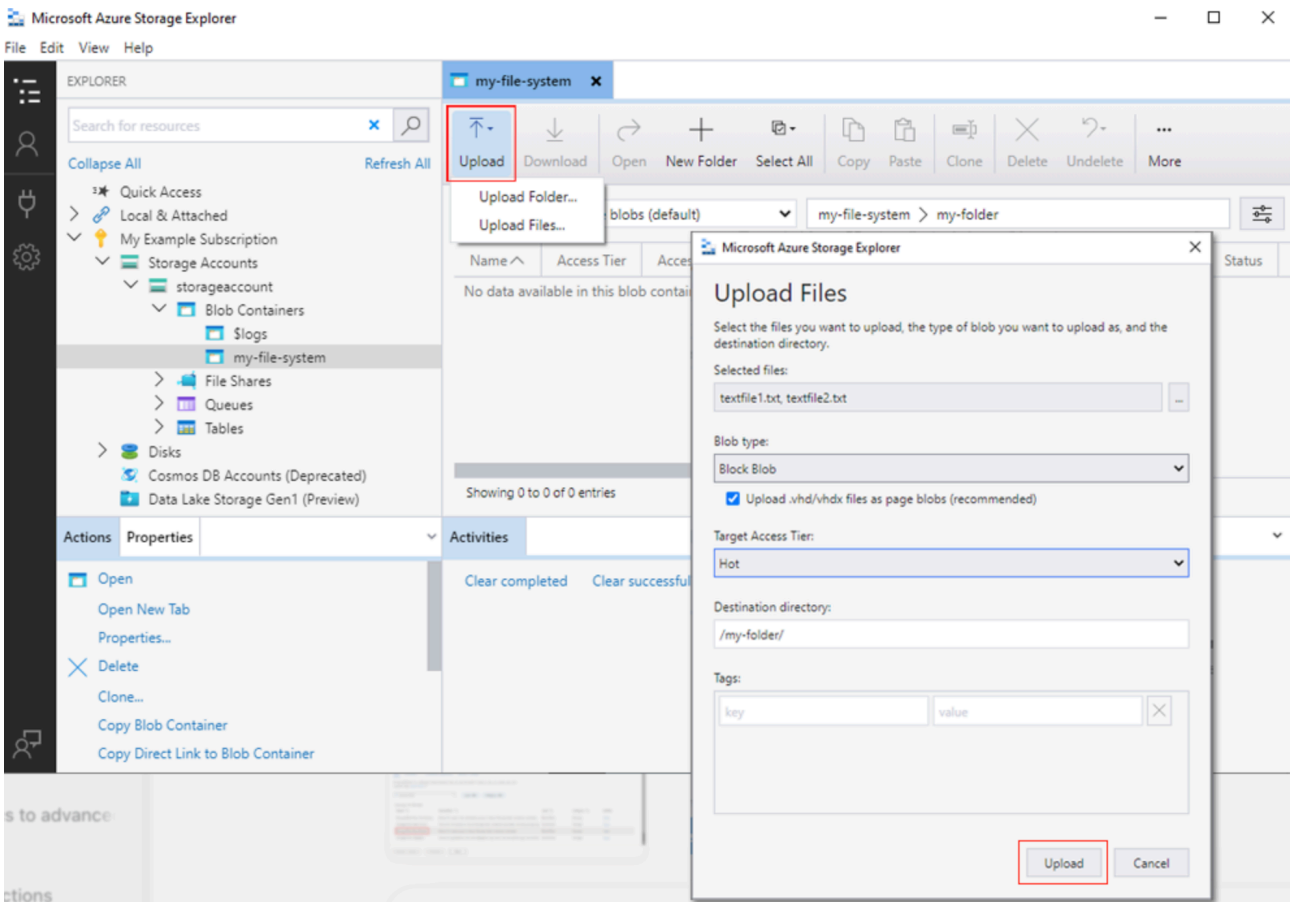
a. Configure or describe a storage account with hierarchical namespace enabled for Azure Data Lake Storage Gen2.

ANS :

Create a standard storage account and enable the **Hierarchical Namespace** option under the "Advanced" tab. This upgrades the account to ADLS Gen2, enabling directory-level ACLs.

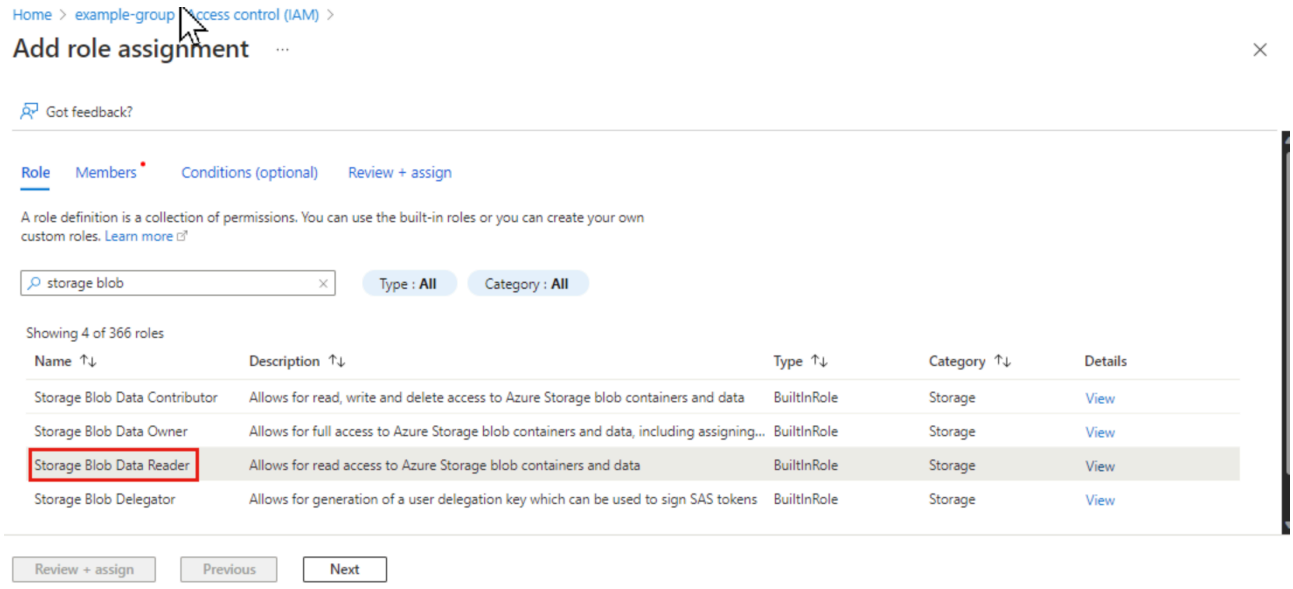


b. Create a filesystem or container and upload a sample dataset.

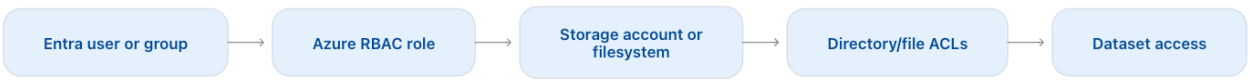


ANS :
Use the Azure CLI or Storage Explorer to create a container (filesystem) and upload a file: az storage fs file upload --account-name <name> --file-system analytics --source data.csv --path raw/data.csv

c. Explain how Microsoft Entra ID and Azure role-based access control
can manage access to the data.
ANS :
Access is managed by assigning roles (like **Storage Blob Data Reader**) to Microsoft Entra ID users or groups. This ensures only authenticated identities can access the data, rather than relying on shared access keys.



d. Explain how you would verify that unauthorized users cannot access the dataset.
ANS :
Log in as a user who has not been assigned an RBAC role for the storage account. Attempt to list or download the file using Azure Storage Explorer or the CLI; the operation should fail with an AuthorizationPermissionMismatch error.



Azure VM troubleshooting

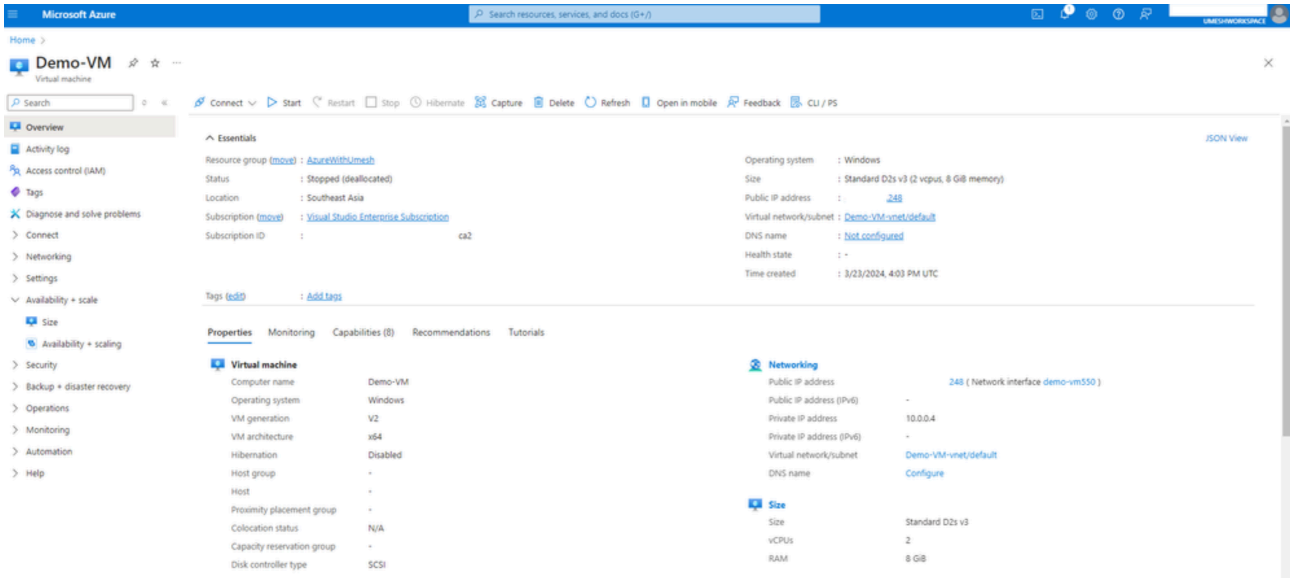
If administrators cannot connect to an Azure VM, check its status, network rules, subnet and routes, then use a secure access method.

a.Verify the VM's provisioning and power states.
ANS :
In the Azure portal, open **Virtual machines** → select the VM → **Overview**.

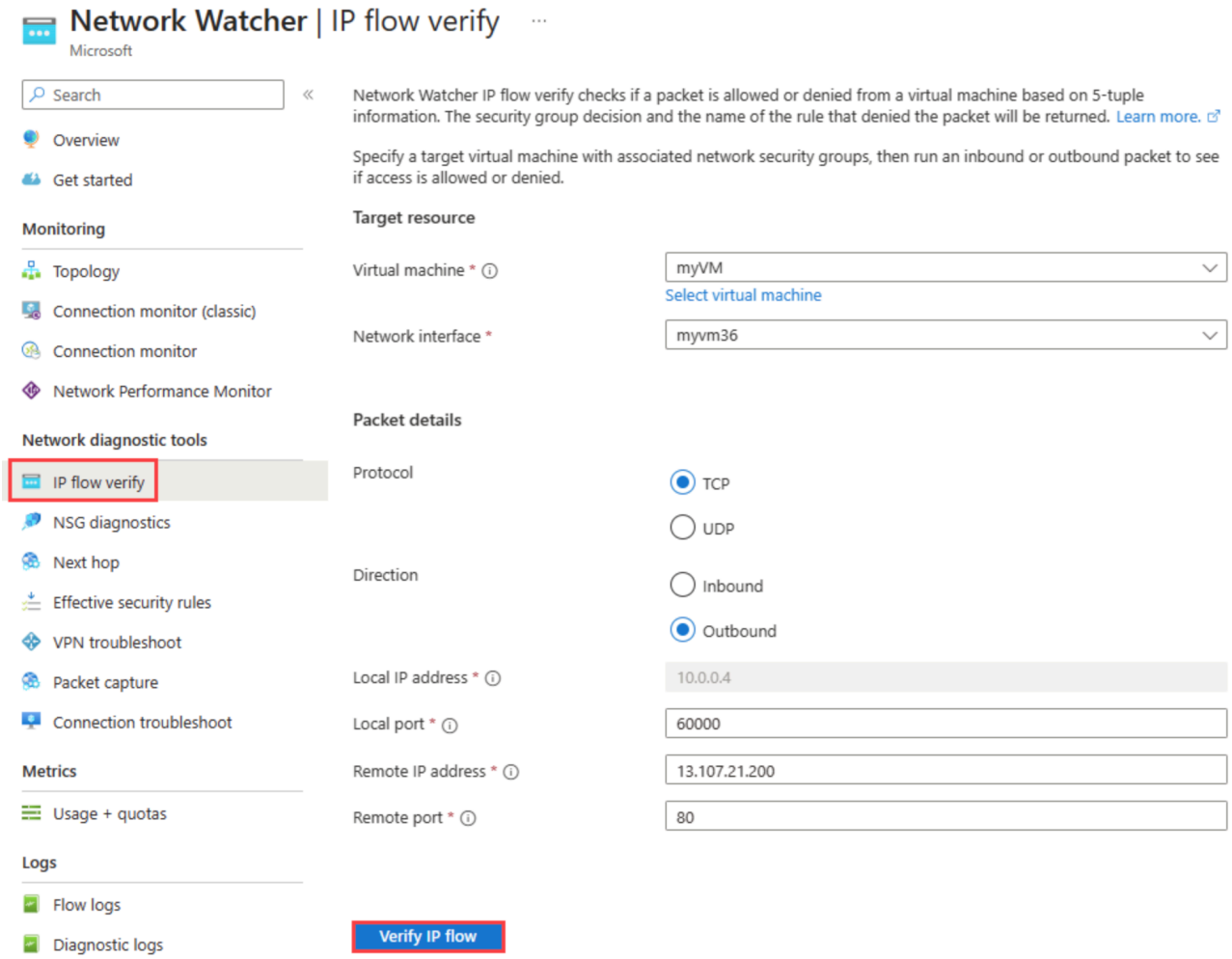
Provisioning state should be **Succeeded**. A failed provisioning state can mean the VM deployment or a recent configuration change did not complete.

Power state should be **Running**. If it is stopped or deallocated, start it and retry.

Check **Activity log** for recent VM, NIC, or network changes that may explain the problem.



b.Explain how Network Security Group rules can prevent connectivity.
ANS :
An NSG can block a connection if it lacks an applicable allow rule or has a higher-priority deny rule. Check NSGs attached to both the **subnet** and the VM's **network interface**.



For example, an administrator connecting over SSH needs an inbound TCP rule for port **22**; RDP typically uses TCP port **3389**. Confirm the rule's source, destination, protocol, port, and priority. Restrict the source to the administrator's known IP range; don't open the port to `Any` or `0.0.0.0/0`.

c. Explain how subnet and routing configuration affect connectivity.

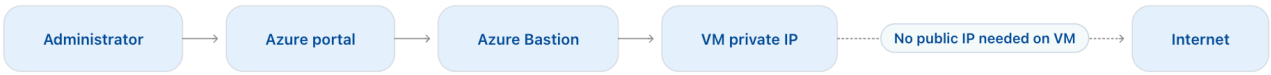
ANS :

Confirm the VM's NIC is attached to the intended subnet and that the subnet has the expected address range and connectivity. Inspect the subnet's route table and effective routes: a user-defined route or network appliance may send traffic somewhere unexpected or prevent the return path. Also check peering, VPN, or ExpressRoute connectivity if administrators connect from another network.

d. Recommend a secure troubleshooting method that avoids unnecessarily exposing the VM to the public internet.

ANS :

Prefer **Azure Bastion** for interactive SSH or RDP access. Bastion connects through the Azure portal and does not require a public IP on the VM or inbound SSH/RDP access from the internet. Restrict Bastion access to authorized administrators.For diagnosis, use **Network Watcher → Connection troubleshoot** to test connectivity from a source to the VM, and **IP flow verify** to see whether an NSG rule allows or denies traffic. If available for your environment, use **just-in-time VM access** to open a tightly scoped management port temporarily, with a limited source IP and duration.

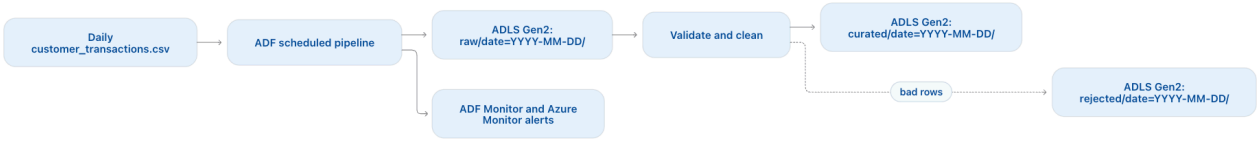


Azure data integration solution

Use Azure Data Factory (ADF) to orchestrate ingestion, Azure Data Lake Storage Gen2 (ADLS Gen2) as the landing and curated storage, and an ADF Mapping Data Flow to validate and clean the CSV

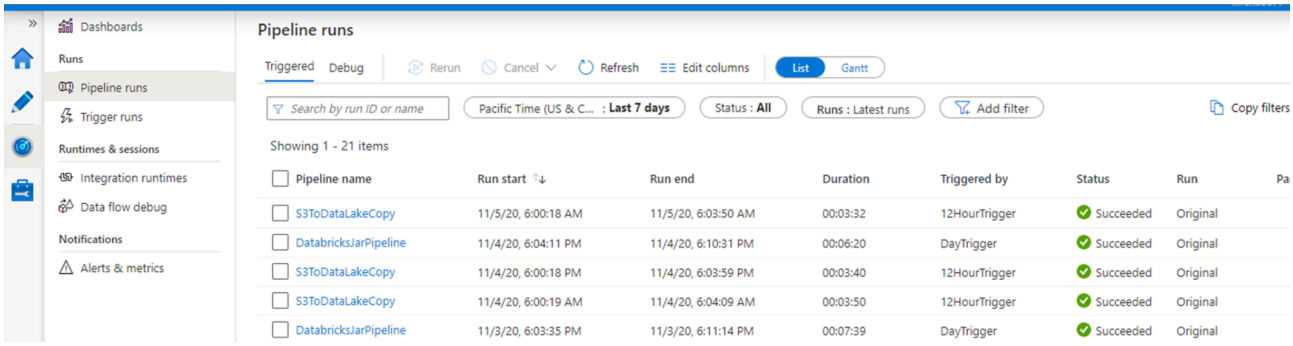
a. Orchestrate data movement

Create an **ADF pipeline** with a daily schedule trigger. Use a **Copy Data** activity to move the CSV, then a **Mapping Data Flow** activity for validation and cleanup.



b. Copy the CSV to ADLS Gen2

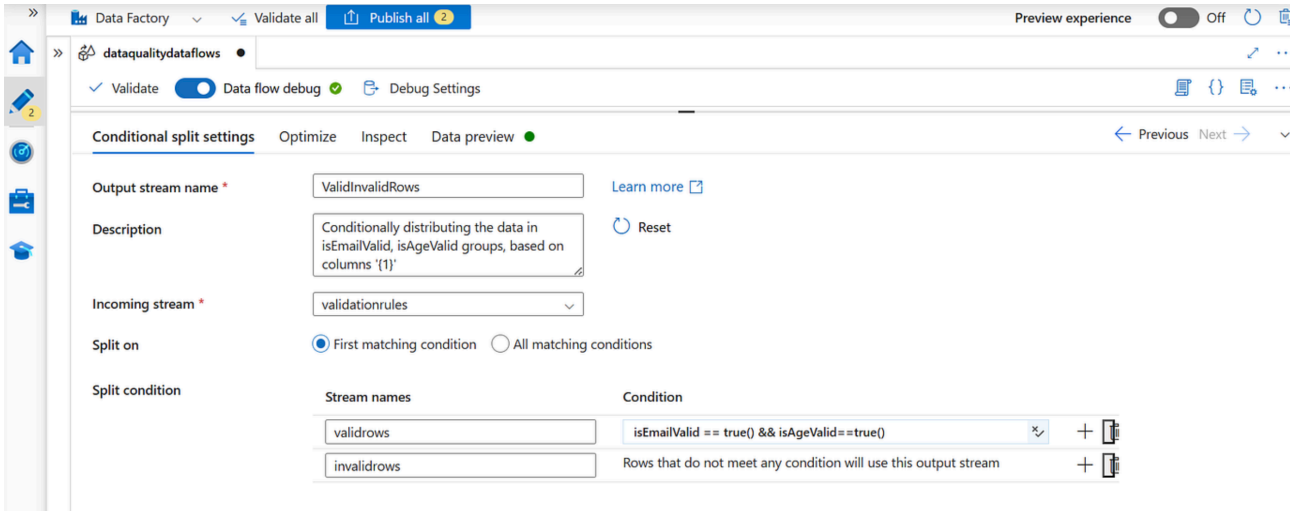
In ADF Studio, create linked services for the source location and ADLS Gen2, then datasets for the CSV and lake destination. Configure the Copy activity to read the incoming file and write it to the lake. Use **Managed Identity** for ADF's access to ADLS, granting it only the required data role



at the narrowest suitable scope.

c. Organize by ingestion date

Parameterize the destination folder path and use the pipeline run date, for example:
Keep the original file in `raw` for traceability; write cleaned data to a separate `curated` path.



d. Handle malformed records and missing values

In a Mapping Data Flow, validate column types and required fields. Send valid rows to `curated` and invalid rows to a `rejected` folder with an error reason. For missing values, apply agreed rules: reject rows missing required fields such as transaction ID, or fill optional fields with a documented default. ADF Copy activity can also skip and log incompatible rows, but a separate rejected-data path makes review and correction easier. ADF Copy activity fault tolerance · Mapping Data Flow Assert transformation

e. Monitor execution and validate completeness

Use ADF's Monitor view to inspect pipeline and activity status, duration, and errors. Set an Azure Monitor alert for failed pipeline runs. Compare source and destination row counts; confirm:Also check that the expected date folder and file exist, and review rejected rows and Copy activity logs before marking the daily load complete. Monitor Azure Data Factory